

Generic Construction of Dual-Server Public Key Authenticated Encryption with Keyword Search

Keita Emura^{*1,2}

¹Kanazawa University, Ishikawa, Japan

²National Institute of Advanced Industrial Science and Technology, Tokyo, Japan

May 7, 2025

Abstract

Chen et al. (IEEE Transactions on Cloud Computing 2022) introduced dual-server public key authenticated encryption with keyword search (DS-PAEKS), and proposed a DS-PAEKS scheme under the decisional Diffie-Hellman assumption. In this paper, we propose a generic construction of DS-PAEKS from PAEKS, public key encryption, and signatures. By providing a concrete attack, we show that the DS-PAEKS scheme of Chen et al. is vulnerable. That is, the proposed generic construction yields the first DS-PAEKS schemes. Our attack with a slight modification works against the Chen et al. dual-server public key encryption with keyword search (DS-PEKS) scheme (IEEE Transactions on Information Forensics and Security 2016). Moreover, we demonstrate that the Tso et al. generic construction of DS-PEKS from public key encryption (IEEE Access 2020) is also vulnerable.

1 Introduction

Public key encryption with keyword search (PEKS) [2] provides a search functionality over encrypted data in a public key setting. A sender encrypts a keyword kw using the public key of a receiver. The receiver then generates a trapdoor for a keyword kw' using the secret key of the receiver. The test algorithm that takes a ciphertext and a trapdoor as input outputs 1 if $kw = kw'$. Similar to correctness, (computational) consistency is defined, where no probabilistic polynomial-time (PPT) adversary can produce kw and kw' such that $kw \neq kw'$ and the test algorithm outputs 1 with a ciphertext of kw and a trapdoor of kw' . It is required that no information about keywords is revealed from ciphertexts. However, information about which keyword is associated with the trapdoor is leaked by running a test algorithm with self-made ciphertexts. Anyone can generate a ciphertext; hence, the keyword guessing attack is unavoidable in PEKS. To prevent the keyword guessing attack, public key authenticated encryption with keyword search (PAEKS) [7, 9–16] has been proposed, where a sender secret key is required for encryption. PAEKS requires that no information about the keyword is leaked from both ciphertexts and trapdoors.¹

Chen et al. [4] further extended PAEKS by introducing a dual-server setting, which they call dual-server PAEKS (DS-PAEKS). In DS-PAEKS, there are two servers, the assistant server and the

^{*}k-emura@se.kanazawa-u.ac.jp

¹As another way to prevent the keyword guessing attack is so-called designated-tester PEKS (e.g., [17]) where a test server has a public and secret key pair, and the test algorithm takes the server's secret key as input.

test server that manage their own public and secret keys, respectively. DS-PAEKS can be regarded as an extension of dual-server PEKS (DS-PEKS) [5] which does not require the secret key of the sender for encryption. The DS-PAEKS flow is described below.

- A sender encrypts a keyword kw using the secret key of the sender sk_S and the public keys of a receiver pk_R , assistant server pk_{AS} , and test server pk_{TS} and uploads the ciphertext $ct_{DS-PAEKS}$ to the assistant server.
- A receiver generates a trapdoor $td_{kw'}$ for a keyword kw' using the secret key of the receiver sk_R and the public keys of a sender pk_S , assistant server pk_{AS} , and test server pk_{TS} , and uploads $td_{kw'}$ to the assistant server.
- The assistant server converts the ciphertext and the trapdoor to an intermediate ciphertext $int-ct_{DS-PAEKS}$ via the transition algorithm using the secret key of the assistant server sk_{AS} , and sends $int-ct_{DS-PAEKS}$ to the test server.
- Finally, the test server runs the test algorithm, that takes the intermediate ciphertext $int-ct_{DS-PAEKS}$ and the secret key of the test server sk_{TS} as input.

Chen et al. claimed that the dual-server setting prevents running the test algorithm by a single server that prevents a keyword guessing attack. That is, in PAEKS, the cases that an adversary trivially wins are excluded in the security definitions, and thus, if a server that runs the test algorithm obtains a ciphertext and a trapdoor of the challenge keyword for the same sender, then there is no way to prevent keyword guessing attacks in PAEKS. By introducing dual servers, there is room for protecting keyword guessing attacks in more strict way. For example, for an adversary that is modeled as a malicious assistant server, it is guaranteed that no information about the keyword is leaked from the challenge ciphertext, even if the adversary obtains a trapdoor for the challenge keyword, and converts the challenge ciphertext and the trapdoor. Similarly, for an adversary that is modeled as a malicious test server, it is guaranteed that no information about the keyword is leaked from the challenge ciphertext, even if the adversary obtains the corresponding intermediate ciphertext converted from the challenge ciphertext and trapdoor. Here, it is assumed that two servers do not collude, i.e., an adversary has a secret key of either the assistant server or the test server in the security models.

Our Contribution. In this paper, we propose a generic construction of DS-PAEKS derived from PAEKS, two PKE schemes, and two signature schemes. As concrete instantiations of the proposed generic construction, with appropriate PKE and signature schemes, we can employ PAEKS schemes with several complexity assumptions instantiated by the Li-Boyen generic construction [12], e.g., pairings, lattices, isogenies, factoring, and so on.

We also give a concrete attack against the Chen et al. DS-PAEKS scheme [4]. We emphasize that the attack is done according to their security model where an adversary obtains the assistant server's secret key sk_{AS} and is allowed to access the test oracle. That is, the proposed generic construction yields the first DS-PAEKS schemes. Our attack with a slight modification works against the Chen et al. DS-PEKS scheme [5]. Moreover, we demonstrate that a generic construction of DS-PEKS from PKE [18] is also vulnerable.

Remark. Cheng and Meng [8] proposed server-aided PAEKS (SA-PAEKS). Though SA-PAEKS also introduces two servers, the roles of these servers are different from those of DS-PAEKS. They focused the fact that both ciphertext and trapdoor are associated with a sender and a receiver in PAEKS. That is, a trapdoor is designated to a sender and the receiver needs to prepare trapdoors for all senders even the receiver would like to search one keyword. Thus, the test server needs to

run the test algorithm in several times. To reduce the costs, they introduced a sender server and a receiver server (in addition to the test server) and reduced the complex “multi-senders-to-multi-receivers” case to a simple “single-sender-server-to-single-receiver-server” case. The main purpose of SA-PAEKS is to reduce the cost whereas the main purpose of DS-PAEKS is to protect keyword guessing attacks in more strict way.

2 Preliminaries: PAEKS, PKE, and Signatures

Definition 1 (Syntax of PAEKS). *A PAEKS scheme PAEKS consists of the six algorithms (PAEKS.Setup, PAEKS.KG_R, PAEKS.KG_S, PAEKS.Enc, PAEKS.Trapdoor, PAEKS.Test) defined as follows.*

PAEKS.Setup: *The setup algorithm takes a security parameter λ as input, and outputs a common parameter pp . We assume that pp implicitly contains the keyword space $\mathcal{KS}$.*

PAEKS.KG_R: *The receiver key generation algorithm takes pp as input, and outputs a public key pk_R and secret key sk_R .*

PAEKS.KG_S: *The sender key generation algorithm takes pp as input, and outputs a public key pk_S and secret key sk_S .*

PAEKS.Enc: *The keyword encryption algorithm takes pk_R , pk_S , sk_S , and a keyword $kw \in \mathcal{KS}$ as input, and outputs a ciphertext ct_{PAEKS} .*

PAEKS.Trapdoor: *The trapdoor algorithm takes pk_R , pk_S , sk_R , and a keyword $kw' \in \mathcal{KS}$ as input, and outputs a trapdoor $\text{td}_{kw'}$.*

PAEKS.Test: *The test algorithm takes ct_{PAEKS} and $\text{td}_{kw'}$ as input, and outputs 1 or 0.*

Definition 2 (Correctness). *For any security parameter λ , any common parameter $\text{pp} \leftarrow \text{PAEKS.Setup}(1^\lambda)$, any key pair $(\text{pk}_R, \text{sk}_R) \leftarrow \text{PAEKS.KG}_R(\text{pp})$ and $(\text{pk}_S, \text{sk}_S) \leftarrow \text{PAEKS.KG}_S(\text{pp})$, and any keyword $kw \in \mathcal{KS}$, let $\text{ct}_{\text{PAEKS}} \leftarrow \text{PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, kw)$ and $\text{td}_{kw} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, kw)$. Then $\Pr[\text{PAEKS.Test}(\text{ct}_{\text{PAEKS}}, \text{td}_{kw}) = 1] = 1 - \text{negl}(\lambda)$ holds.*

Definition 3 (Computational Consistency). *We define the experiment:*

$\text{Exp}_{\text{PAEKS}, \mathcal{A}}^{\text{consist}}(\lambda) :$
 $\text{pp} \leftarrow \text{PAEKS.Setup}(1^\lambda)$
 $(\text{pk}_R, \text{sk}_R) \leftarrow \text{PAEKS.KG}_R(\text{pp}); (\text{pk}_S, \text{sk}_S) \leftarrow \text{PAEKS.KG}_S(\text{pp})$
 $(kw, kw') \leftarrow \mathcal{A}(\text{pp}, \text{pk}_R, \text{pk}_S)$ s.t. $kw, kw' \in \mathcal{KS} \wedge kw \neq kw'$
 $\text{ct}_{\text{PAEKS}} \leftarrow \text{PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, kw)$
 $\text{td}_{kw'} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, kw')$
If $\text{PAEKS.Test}(\text{ct}_{\text{PAEKS}}, \text{td}_{kw'}) = 1$, then output 1, and 0 otherwise.

PAEKS is consistent if the advantage

$$\text{Adv}_{\text{PAEKS}, \mathcal{A}}^{\text{consist}}(\lambda) := \Pr[\text{Exp}_{\text{PAEKS}, \mathcal{A}}^{\text{consist}}(\lambda) = 1]$$

is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Next, we define the indistinguishability against the chosen keyword attack (IND-CKA) and that against the inside keyword guessing attack (IND-IKGA), which ensure that no information about the keyword is leaked from ciphertexts or trapdoors, respectively.

Definition 4 (IND-CKA). *We define the experiment:*

$\text{Exp}_{\text{PAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda) :$
 $\text{pp} \leftarrow \text{PAEKS.Setup}(1^\lambda)$
 $(\text{pk}_R, \text{sk}_R) \leftarrow \text{PAEKS.KG}_R(\text{pp}); (\text{pk}_S, \text{sk}_S) \leftarrow \text{PAEKS.KG}_S(\text{pp})$
 $(kw_0^*, kw_1^*, \text{state}) \leftarrow \mathcal{A}^\mathcal{O}(\text{pp}, \text{pk}_R, \text{pk}_S) \text{ s.t. } kw_0^*, kw_1^* \in \mathcal{KS} \wedge kw_0^* \neq kw_1^*$
 $\beta \xleftarrow{\$} \{0, 1\}; \text{ct}_{\text{PAEKS}}^* \leftarrow \text{PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, kw_\beta^*)$
 $\beta' \leftarrow \mathcal{A}^\mathcal{O}(\text{state}, \text{ct}_{\text{PAEKS}}^*)$
If $\beta = \beta'$ then output 1, and 0 otherwise.

Here, $\mathcal{O} := \{\mathcal{O}_C(\cdot), \mathcal{O}_{\text{Trap}}(\cdot)\}$. $\mathcal{O}_C$ takes $kw \in \mathcal{KS}$ as input, and returns the result of $\text{PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, kw)$. Here, there is no restriction. $\mathcal{O}_{\text{Trap}}$ takes $kw' \in \mathcal{KS}$ as input, and returns the result of $\text{PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, kw')$. Here, $kw' \notin \{kw_0^*, kw_1^*\}$. PAEKS is IND-CKA secure if the advantage

$$\text{Adv}_{\text{PAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda) := |\Pr[\text{Exp}_{\text{PAEKS}, \mathcal{A}}^{\text{IND-CKA}}(\lambda) = 1] - 1/2|$$

is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Definition 5 (IND-IKGA). *We define the experiment:*

$\text{Exp}_{\text{PAEKS}, \mathcal{A}}^{\text{IND-IKGA}}(\lambda) :$
 $\text{pp} \leftarrow \text{PAEKS.Setup}(1^\lambda)$
 $(\text{pk}_R, \text{sk}_R) \leftarrow \text{PAEKS.KG}_R(\text{pp}); (\text{pk}_S, \text{sk}_S) \leftarrow \text{PAEKS.KG}_S(\text{pp})$
 $(kw_0^*, kw_1^*, \text{state}) \leftarrow \mathcal{A}^\mathcal{O}(\text{pp}, \text{pk}_R, \text{pk}_S) \text{ s.t. } kw_0^*, kw_1^* \in \mathcal{KS} \wedge kw_0^* \neq kw_1^*$
 $\beta \xleftarrow{\$} \{0, 1\}; \text{td}_{kw_\beta^*}^* \leftarrow \text{PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, kw_\beta^*)$
 $\beta' \leftarrow \mathcal{A}^\mathcal{O}(\text{state}, \text{td}_{kw_\beta^*}^*)$
If $\beta = \beta'$ then output 1, and 0 otherwise.

Here, $\mathcal{O} := \{\mathcal{O}_C(\cdot), \mathcal{O}_{\text{Trap}}(\cdot)\}$. $\mathcal{O}_C$ takes $kw \in \mathcal{KS}$ as input, and returns the result of $\text{PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, kw)$. Here, $kw \notin \{kw_0^*, kw_1^*\}$. $\mathcal{O}_{\text{Trap}}$ takes $kw' \in \mathcal{KS}$ as input, and returns the result of $\text{PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, kw')$. Here $kw' \notin \{kw_0^*, 0, kw_1^*, 0\}$. PAEKS is IND-IKGA secure if the advantage

$$\text{Adv}_{\text{PAEKS}, \mathcal{A}}^{\text{IND-IKGA}}(\lambda) := |\Pr[\text{Exp}_{\text{PAEKS}, \mathcal{A}}^{\text{IND-IKGA}}(\lambda) = 1] - 1/2|$$

is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Finally, we give definitions of PKE and signatures.

PKE. Let $\text{PKE} = (\text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a PKE scheme. The key generation algorithm PKE.KeyGen takes a security parameter λ as input, and outputs a key pair (PK, DK) . The encryption algorithm PKE.Enc takes PK and a plaintext M , and outputs a ciphertext C . The decryption algorithm PKE.Dec takes DK and C , and outputs M or $\perp$. We require that PKE provides indistinguishability against the chosen-ciphertext attack (IND-CCA), where an PPT adversary $\mathcal{A}$

is allowed to issue decryption queries $C \neq C^*$ where C^* is the challenge ciphertext that is an encryption of either M_0^* or M_1^* . $\mathcal{A}$ wins if $\mathcal{A}$ can distinguish whether C^* is an encryption of M_0^* or M_1^* .

Signature. Let $\text{Sig} = (\text{Sig.KeyGen}, \text{Sign}, \text{Verify})$ be a signature scheme. The key generation algorithm Sig.KeyGen takes a security parameter λ , and outputs a key pair (vk, sigk) . The signing algorithm Sign takes sigk and a message M as input, and outputs a signature σ . Here, we explicitly assume that the Sign algorithm is probabilistic (See the proof of Lemma 3). The verification algorithm Verify takes vk , σ , and M as input, and outputs 0 or 1. We require that Sig provides strongly existential unforgeability under the adaptive chosen message attack (sEUF-CMA), where a PPT adversary $\mathcal{A}$ is allowed to issue a signing query M and obtains $\sigma \leftarrow \text{Sign}(\text{sigk}, M)$. (M, σ) is then preserved to a set Set . $\mathcal{A}$ wins if $\mathcal{A}$ can produce (M^*, σ^*) , where $\text{Verify}(\text{vk}, \sigma^*, M^*) = 1$ and $(M^*, \sigma^*) \notin \text{Set}$.

3 Definitions of DS-PAEKS

In this section, we introduce the DS-PAEKS definitions. Figure 1 describes the DS-PAEKS flow.

Definition 6 (Syntax of DS-PAEKS). *A DS-PAEKS scheme DS-PAEKS consists of the nine algorithms (DS-PAEKS.Setup, DS-PAEKS.KG_R, DS-PAEKS.KG_S, DS-PAEKS.KG_{AS}, DS-PAEKS.KG_{TS}, DS-PAEKS.Enc, DS-PAEKS.Trapdoor, DS-PAEKS.Transition, DS-PAEKS.Test) defined as follows.*

DS-PAEKS.Setup: *The setup algorithm takes a security parameter λ as input, and outputs a common parameter pp . We assume that pp implicitly contains the keyword space KS .*

DS-PAEKS.KG_R: *The receiver key generation algorithm takes pp as input, and outputs a public key pk_R and secret key sk_R .*

DS-PAEKS.KG_S: *The sender key generation algorithm takes pp as input, and outputs a public key pk_S and secret key sk_S .*

DS-PAEKS.KG_{AS}: *The assistant server key generation algorithm takes pp as input, and outputs a public key pk_{AS} and secret key sk_{AS} .*

DS-PAEKS.KG_{TS}: *The test server key generation algorithm takes pp as input, and outputs a public key pk_{TS} and secret key sk_{TS} .*

DS-PAEKS.Enc: *The keyword encryption algorithm takes pk_R , pk_S , sk_S , pk_{AS} , pk_{TS} , and a keyword $kw \in KS$ as input, and outputs a ciphertext $\text{ct}_{\text{DS-PAEKS}}$.*

DS-PAEKS.Trapdoor: *The trapdoor algorithm takes pk_R , pk_S , sk_R , pk_{AS} , pk_{TS} , and a keyword $kw' \in KS$ as input, and outputs a trapdoor $\text{td}_{\text{DS-PAEKS}}$.*

DS-PAEKS.Transition: *The transition algorithm takes pk_R , pk_S , pk_{AS} , sk_{AS} , $\text{ct}_{\text{DS-PAEKS}}$, and $\text{td}_{\text{DS-PAEKS}}$ as input, and outputs an intermediate ciphertext $\text{int-ct}_{\text{DS-PAEKS}}$.*

DS-PAEKS.Test: *The test algorithm takes pk_R , pk_S , pk_{TS} , sk_{TS} , and $\text{int-ct}_{\text{DS-PAEKS}}$ as input, and outputs 1 or 0.*

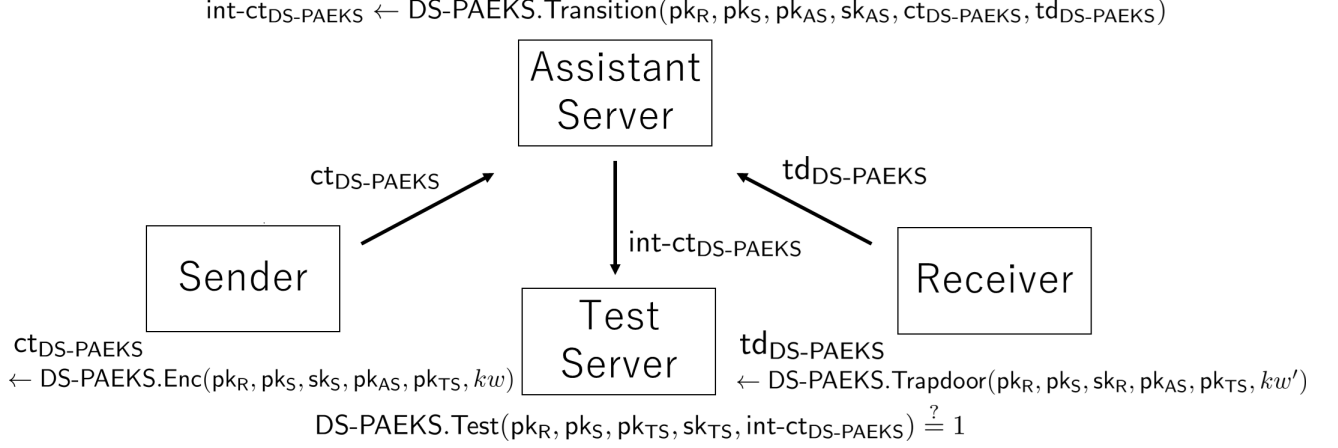


Figure 1: DS-PAEKS

Definition 7 (Correctness). *For any security parameter λ , any common parameter $\text{pp} \leftarrow \text{DS-PAEKS.Setup}(1^\lambda)$, any key pair $(\text{pk}_R, \text{sk}_R) \leftarrow \text{DS-PAEKS.KG}_R(\text{pp})$, $(\text{pk}_S, \text{sk}_S) \leftarrow \text{DS-PAEKS.KG}_S(\text{pp})$, $(\text{pk}_{AS}, \text{sk}_{AS}) \leftarrow \text{DS-PAEKS.KG}_{AS}(\text{pp})$, and $(\text{pk}_{TS}, \text{sk}_{TS}) \leftarrow \text{DS-PAEKS.KG}_{TS}(\text{pp})$, and any keyword $kw \in \mathcal{KS}$, let $\text{ct}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{AS}, \text{pk}_{TS}, kw)$ and $\text{td}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, \text{pk}_{AS}, \text{pk}_{TS}, kw')$. Then, $\Pr[\text{DS-PAEKS.Test}(\text{pk}_R, \text{pk}_S, \text{pk}_{TS}, \text{sk}_{TS}, \text{int-ct}_{\text{DS-PAEKS}}) = 1] = 1 - \text{negl}(\lambda)$ holds for $\text{int-ct}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Transition}(\text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{sk}_{AS}, \text{ct}_{\text{DS-PAEKS}}, \text{td}_{\text{DS-PAEKS}})$.*

Definition 8 (Computational Consistency). *We define the experiment:*

$\text{Exp}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{consist}}(\lambda) :$

$\text{pp} \leftarrow \text{DS-PAEKS.Setup}(1^\lambda)$

$(\text{pk}_R, \text{sk}_R) \leftarrow \text{DS-PAEKS.KG}_R(\text{pp}); (\text{pk}_S, \text{sk}_S) \leftarrow \text{DS-PAEKS.KG}_S(\text{pp})$

$(\text{pk}_{AS}, \text{sk}_{AS}) \leftarrow \text{DS-PAEKS.KG}_{AS}(\text{pp}); (\text{pk}_{TS}, \text{sk}_{TS}) \leftarrow \text{DS-PAEKS.KG}_{TS}(\text{pp})$

$(kw, kw') \leftarrow \mathcal{A}(\text{pp}, \text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{pk}_{TS})$ s.t. $kw, kw' \in \mathcal{KS} \wedge kw \neq kw'$

$\text{ct}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{AS}, \text{pk}_{TS}, kw)$

$\text{td}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, \text{pk}_{AS}, \text{pk}_{TS}, kw')$

$\text{int-ct}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Transition}(\text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{sk}_{AS}, \text{ct}_{\text{DS-PAEKS}}, \text{td}_{\text{DS-PAEKS}})$

If $\text{DS-PAEKS.Test}(\text{pk}_R, \text{pk}_S, \text{pk}_{TS}, \text{sk}_{TS}, \text{int-ct}_{\text{DS-PAEKS}}) = 1$,

then output 1, and 0 otherwise.

We say that DS-PAEKS is consistent if the advantage

$$\text{Adv}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{consist}}(\lambda) := \Pr[\text{Exp}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{consist}}(\lambda) = 1]$$

is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Chen et al. gave a formal security definition of DS-PAEKS. However, the following restrictions in their security definitions can be observed: An adversary that is modeled as a malicious assistant server is allowed to issue any query, including challenge keywords, to the encryption, trapdoor, and test oracles. Constructing a DS-PAEKS scheme, which is secure in this definition, is impossible because of the following general attack: An adversary that has the secret key of the assistant server sk_{AS} issues a challenge keyword kw_0^* to the trapdoor oracle. After obtaining the challenge ciphertext $\text{ct}_{\text{DS-PAEKS}}^*$, the adversary prepares an intermediate ciphertext $\text{int-ct}_{\text{DS-PAEKS}}$ from $\text{ct}_{\text{DS-PAEKS}}^*$,

trapdoor for kw_0^* $\text{td}_{\text{DS-PAEKS}}$, and sk_{AS} , and sends $\text{int-ct}_{\text{DS-PAEKS}}$ to the test oracle. If $\text{ct}_{\text{DS-PAEKS}}^*$ is an encryption of kw_0^* , then the test oracle returns 1, and 0 if $\text{ct}_{\text{DS-PAEKS}}^*$ is an encryption of kw_1^* . This completely breaks the security. Thus, we newly introduce the DS-PAEKS definitions. We note that our attack against the Chen et al. DS-PAEKS scheme given in this paper is not the above general attack.

First, we define IND-CKA for the assistant server (IND-AS-CKA), where the adversary is given sk_{AS} . Considering the role of the assistant server, we must guarantee that no information about the keyword is leaked from the challenge ciphertext, even if the adversary obtains a trapdoor for the challenge keyword, and runs the DS-PAEKS.Transition algorithm with the challenge ciphertext and the trapdoor. However, if there is no restriction, then the adversary can trivially break the IND-AS-CKA security, i.e., by using sk_{AS} , the adversary generates an intermediate ciphertext from the challenge ciphertext and a trapdoor of either kw_0^* or kw_1^* , and sends the intermediate ciphertext to the test oracle $\mathcal{O}_{\text{Test}}$. Thus, we introduce the following restriction: the adversary is allowed to issue $\text{int-ct}_{\text{DS-PAEKS}}$ to $\mathcal{O}_{\text{Test}}$ where $\text{int-ct}_{\text{DS-PAEKS}} \notin \{\text{int-ct}_{\text{DS-PAEKS}} \mid \text{int-ct}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Transition}(\text{pk}_{\text{AS}}, \text{sk}_{\text{AS}}, \text{ct}_{\text{DS-PAEKS}}^*, \text{td}_{\text{DS-PAEKS}}) \wedge \text{td}_{\text{DS-PAEKS}} \in \text{TSet}\}$. Here, TSet is a set of trapdoors for the challenge keywords kw_0^* and kw_1^* . We remark that kw_0^* and kw_1^* are declared during the challenge phase. Thus, TSet is defined after the challenge phase.

Definition 9 (IND-AS-CKA). *We define the experiment:*

$\text{Exp}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{IND-AS-CKA}}(\lambda) :$

$\text{pp} \leftarrow \text{DS-PAEKS.Setup}(1^\lambda)$

$(\text{pk}_R, \text{sk}_R) \leftarrow \text{DS-PAEKS.KG}_R(\text{pp}); (\text{pk}_S, \text{sk}_S) \leftarrow \text{DS-PAEKS.KG}_S(\text{pp})$

$(\text{pk}_{\text{AS}}, \text{sk}_{\text{AS}}) \leftarrow \text{DS-PAEKS.KG}_{\text{AS}}(\text{pp}); (\text{pk}_{\text{TS}}, \text{sk}_{\text{TS}}) \leftarrow \text{DS-PAEKS.KG}_{\text{TS}}(\text{pp})$

$\text{TSet} := \emptyset$

$(kw_0^*, kw_1^*, \text{state}) \leftarrow \mathcal{A}^\mathcal{O}(\text{pp}, \text{pk}_R, \text{pk}_S, \text{pk}_{\text{AS}}, \text{sk}_{\text{AS}}, \text{pk}_{\text{TS}})$

s.t. $kw_0^*, kw_1^* \in \mathcal{KS} \wedge kw_0^* \neq kw_1^*$

$\beta \xleftarrow{\$} \{0, 1\}; \text{ct}_{\text{DS-PAEKS}}^* \leftarrow \text{DS-PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{\text{AS}}, \text{pk}_{\text{TS}}, kw_\beta^*)$

$\beta' \leftarrow \mathcal{A}^\mathcal{O}(\text{state}, \text{ct}_{\text{DS-PAEKS}}^*)$

If $\beta = \beta'$ then output 1, and 0 otherwise.

Here, $\mathcal{O} := \{\mathcal{O}_C(\cdot), \mathcal{O}_{\text{Trap}}(\cdot), \mathcal{O}_{\text{Test}}(\cdot)\}$. $\mathcal{O}_C$ takes $kw \in \mathcal{KS}$ as input, and returns the result of $\text{DS-PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{\text{AS}}, \text{pk}_{\text{TS}}, kw)$. Here, there is no restriction. $\mathcal{O}_{\text{Trap}}$ takes $kw' \in \mathcal{KS}$ as input, and returns $\text{td}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, \text{pk}_{\text{AS}}, \text{pk}_{\text{TS}}, kw')$. Here, there is no restriction. If $kw \in \{kw_0^*, kw_1^*\}$, then $\text{TSet} := \text{TSet} \cup \{\text{td}_{\text{DS-PAEKS}}\}$. We note that in the challenge phase, TSet is updated by the trapdoors of $kw \in \{kw_0^*, kw_1^*\}$ which are generated before $\mathcal{A}$ declares (kw_0^*, kw_1^*) . $\mathcal{O}_{\text{Test}}$ takes $\text{int-ct}_{\text{DS-PAEKS}}$, and returns the result of $\text{DS-PAEKS.Test}(\text{pk}_R, \text{pk}_S, \text{pk}_{\text{TS}}, \text{sk}_{\text{TS}}, \text{int-ct}_{\text{DS-PAEKS}})$. Here, $\text{int-ct}_{\text{DS-PAEKS}} \notin \{\text{int-ct}_{\text{DS-PAEKS}} \mid \text{int-ct}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Transition}(\text{pk}_R, \text{pk}_S, \text{pk}_{\text{AS}}, \text{sk}_{\text{AS}}, \text{ct}_{\text{DS-PAEKS}}^*, \text{td}_{\text{DS-PAEKS}}) \wedge \text{td}_{\text{DS-PAEKS}} \in \text{TSet}\}$. DS-PAEKS is IND-AS-CKA secure if the advantage

$$\text{Adv}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{IND-AS-CKA}}(\lambda) := |\Pr[\text{Exp}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{IND-AS-CKA}}(\lambda) = 1] - 1/2|$$

is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Next, we define IND-CKA for the test server (IND-TS-CKA), where the adversary is given sk_{TS} . Considering the role of the test server, we must guarantee that no information about the keyword

is leaked from the challenge ciphertext, even if the corresponding intermediate ciphertext is given. However, if the adversary is allowed to obtain a trapdoor for the challenge keyword, the adversary can trivially break the IND-TS-CKA security. Thus, we restrict the input of the trapdoor oracle $\mathcal{O}_{\text{Trap}}$ as $kw' \notin \{kw_0^*, kw_1^*\}$.

Definition 10 (IND-TS-CKA). *We define the experiment:*

$\text{Exp}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{IND-TS-CKA}}(\lambda) :$
 $\text{pp} \leftarrow \text{DS-PAEKS.Setup}(1^\lambda)$
 $(\text{pk}_R, \text{sk}_R) \leftarrow \text{DS-PAEKS.KG}_R(\text{pp}); (\text{pk}_S, \text{sk}_S) \leftarrow \text{DS-PAEKS.KG}_S(\text{pp})$
 $(\text{pk}_{AS}, \text{sk}_{AS}) \leftarrow \text{DS-PAEKS.KG}_{AS}(\text{pp}); (\text{pk}_{TS}, \text{sk}_{TS}) \leftarrow \text{DS-PAEKS.KG}_{TS}(\text{pp})$
 $(kw_0^*, kw_1^*, \text{state}) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp}, \text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{pk}_{TS}, \text{sk}_{TS})$
s.t. $kw_0^*, kw_1^* \in \mathcal{KS} \wedge kw_0^* \neq kw_1^*$
 $\beta \xleftarrow{\$} \{0, 1\}; \text{ct}_{\text{DS-PAEKS}}^* \leftarrow \text{DS-PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{AS}, \text{pk}_{TS}, kw_\beta^*)$
 $\beta' \leftarrow \mathcal{A}^{\mathcal{O}}(\text{state}, \text{ct}_{\text{DS-PAEKS}}^*)$
If $\beta = \beta'$ *then output 1, and 0 otherwise.*

Here, $\mathcal{O} := \{\mathcal{O}_C(\cdot), \mathcal{O}_{\text{Trap}}(\cdot), \mathcal{O}_{\text{Trans}}(\cdot, \cdot)\}$. $\mathcal{O}_C$ takes $kw \in \mathcal{KS}$ as input, and returns the result of $\text{DS-PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{AS}, \text{pk}_{TS}, kw)$. Here, there is no restriction. $\mathcal{O}_{\text{Trap}}$ takes $kw' \in \mathcal{KS}$ as input, and returns the result of $\text{DS-PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, \text{pk}_{AS}, \text{pk}_{TS}, kw')$. Here, $kw' \notin \{kw_0^*, kw_1^*\}$. $\mathcal{O}_{\text{Trans}}$ takes $\text{ct}_{\text{DS-PAEKS}}^*$ and $\text{td}_{\text{DS-PAEKS}}$, and returns the result of $\text{DS-PAEKS.Transition}(\text{pk}_{AS}, \text{sk}_{AS}, \text{ct}_{\text{DS-PAEKS}}^*, \text{td}_{\text{DS-PAEKS}})$. Here, there is no restriction. DS-PAEKS is IND-TS-CKA secure if the advantage

$$\text{Adv}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{IND-TS-CKA}}(\lambda) := |\Pr[\text{Exp}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{IND-TS-CKA}}(\lambda) = 1] - 1/2|$$

is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Next, we define indistinguishability against the inside keyword guessing attack (IND-IKGA) which ensures that no information about the keyword is leaked from trapdoors. As in IND-CKA, we separately define the cases where an adversary is modeled as either the assistant server or the test server. First, we define IND-IKGA for the assistant server (IND-AS-IKGA) where the adversary is given sk_{AS} . As in IND-AS-CKA, we define a set of ciphertexts for the challenge keywords CTSet . $\text{CTSet} := \text{CTSet} \cup \{\text{ct}_{\text{DS-PAEKS}}^*\}$ when kw_0^* or kw_1^* is queried to the encryption oracle $\mathcal{O}_C$.

Definition 11 (IND-AS-IKGA). *We define the experiment:*

$\text{Exp}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{IND-AS-IKGA}}(\lambda) :$
 $\text{pp} \leftarrow \text{DS-PAEKS.Setup}(1^\lambda)$
 $(\text{pk}_R, \text{sk}_R) \leftarrow \text{DS-PAEKS.KG}_R(\text{pp}); (\text{pk}_S, \text{sk}_S) \leftarrow \text{DS-PAEKS.KG}_S(\text{pp})$
 $(\text{pk}_{AS}, \text{sk}_{AS}) \leftarrow \text{DS-PAEKS.KG}_{AS}(\text{pp}); (\text{pk}_{TS}, \text{sk}_{TS}) \leftarrow \text{DS-PAEKS.KG}_{TS}(\text{pp})$
 $\text{CTSet} := \emptyset$
 $(kw_0^*, kw_1^*, \text{state}) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp}, \text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{sk}_{AS}, \text{pk}_{TS})$
s.t. $kw_0^*, kw_1^* \in \mathcal{KS} \wedge kw_0^* \neq kw_1^*$
 $\beta \xleftarrow{\$} \{0, 1\}; \text{td}_{\text{DS-PAEKS}}^* \leftarrow \text{DS-PAEKS.Trapdoor}(\text{pk}_R, \text{pk}_S, \text{sk}_R, \text{pk}_{AS}, \text{pk}_{TS}, kw_\beta^*)$
 $\beta' \leftarrow \mathcal{A}^{\mathcal{O}}(\text{state}, \text{td}_{\text{DS-PAEKS}}^*)$
If $\beta = \beta'$ *then output 1, and 0 otherwise.*

Here, $\mathcal{O} := \{\mathcal{O}_C(\cdot), \mathcal{O}_{Trap}(\cdot), \mathcal{O}_{Test}(\cdot)\}$. $\mathcal{O}_C$ takes $kw \in \mathcal{KS}$ as input, and returns $ct_{DS-PAEKS} \leftarrow DS-PAEKS.Enc(pk_R, pk_S, sk_S, pk_{AS}, pk_{TS}, kw)$. Here, there is no restriction. If $kw \in \{kw_0^*, kw_1^*\}$, then $CTSet := CTSet \cup \{ct_{DS-PAEKS}\}$. We note that in the challenge phase, $CTSet$ is updated by the ciphertexts of $kw \in \{kw_0^*, kw_1^*\}$ generated before $\mathcal{A}$ declares (kw_0^*, kw_1^*) . $\mathcal{O}_{Trap}$ takes $kw' \in \mathcal{KS}$ as input, and returns the result of $DS-PAEKS.Trapdoor(pk_R, pk_S, sk_R, pk_{AS}, pk_{TS}, kw')$. Here, $kw' \notin \{kw_0^*, kw_1^*\}$. $\mathcal{O}_{Test}$ takes $int-ct_{DS-PAEKS}$, and returns the result of $DS-PAEKS.Test(pk_R, pk_S, pk_{TS}, sk_{TS}, int-ct_{DS-PAEKS})$. Here, $int-ct_{DS-PAEKS} \notin \{int-ct_{DS-PAEKS} \mid int-ct_{DS-PAEKS} \leftarrow DS-PAEKS.Transition(pk_R, pk_S, pk_{AS}, sk_{AS}, ct_{DS-PAEKS}, td_{DS-PAEKS}^*) \wedge ct_{DS-PAEKS} \in CTSet\}$. $DS-PAEKS$ is *IND-AS-IKGA* secure if the advantage

$$\text{Adv}_{DS-PAEKS, \mathcal{A}}^{\text{IND-AS-IKGA}}(\lambda) := |\Pr[\text{Exp}_{DS-PAEKS, \mathcal{A}}^{\text{IND-AS-IKGA}}(\lambda) = 1] - 1/2|$$

is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Finally, we define *IND-IKGA* security for the test server (*IND-TS-IKGA*), where the adversary is given sk_{TS} .

Definition 12 (*IND-TS-IKGA*). We define the experiment:

$$\begin{aligned} &\text{Exp}_{DS-PAEKS, \mathcal{A}}^{\text{IND-TS-IKGA}}(\lambda) : \\ &\quad pp \leftarrow DS-PAEKS.Setup(1^\lambda) \\ &\quad (pk_R, sk_R) \leftarrow DS-PAEKS.KG_R(pp); (pk_S, sk_S) \leftarrow DS-PAEKS.KG_S(pp) \\ &\quad (pk_{AS}, sk_{AS}) \leftarrow DS-PAEKS.KG_{AS}(pp); (pk_{TS}, sk_{TS}) \leftarrow DS-PAEKS.KG_{TS}(pp) \\ &\quad (kw_0^*, kw_1^*, \text{state}) \leftarrow \mathcal{A}^\mathcal{O}(pp, pk_R, pk_S, pk_{AS}, sk_{AS}, pk_{TS}) \\ &\quad \text{s.t. } kw_0^*, kw_1^* \in \mathcal{KS} \wedge kw_0^* \neq kw_1^* \\ &\quad \beta \xleftarrow{\$} \{0, 1\}; td_{DS-PAEKS}^* \leftarrow DS-PAEKS.Trapdoor(pk_R, pk_S, sk_R, pk_{AS}, pk_{TS}, kw_\beta^*) \\ &\quad \beta' \leftarrow \mathcal{A}^\mathcal{O}(\text{state}, td_{DS-PAEKS}^*) \\ &\quad \text{If } \beta = \beta' \text{ then output 1, and 0 otherwise.} \end{aligned}$$

Here, $\mathcal{O} := \{\mathcal{O}_C(\cdot), \mathcal{O}_{Trap}(\cdot), \mathcal{O}_{Trans}(\cdot, \cdot)\}$. $\mathcal{O}_C$ takes $kw \in \mathcal{KS}$ as input, and returns the result of $DS-PAEKS.Enc(pk_R, pk_S, sk_S, pk_{AS}, pk_{TS}, kw)$. Here, $kw \notin \{kw_0^*, kw_1^*\}$. $\mathcal{O}_{Trap}$ takes $kw' \in \mathcal{KS}$ as input, and returns the result of $DS-PAEKS.Trapdoor(pk_R, pk_S, sk_R, pk_{AS}, pk_{TS}, kw')$. Here, $kw' \notin \{kw_0^*, kw_1^*\}$. $\mathcal{O}_{Trans}$ takes $ct_{DS-PAEKS}$ and $td_{DS-PAEKS}$, and returns the result of $DS-PAEKS.Transition(pk_{AS}, sk_{AS}, ct_{DS-PAEKS}, td_{DS-PAEKS})$. Here, there is no restriction. $DS-PAEKS$ is *IND-TS-IKGA* secure for the test server if the advantage

$$\text{Adv}_{DS-PAEKS, \mathcal{A}}^{\text{IND-TS-IKGA}}(\lambda) := |\Pr[\text{Exp}_{DS-PAEKS, \mathcal{A}}^{\text{IND-TS-IKGA}}(\lambda) = 1] - 1/2|$$

is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

4 Vulnerability of the Chen et al. DS-PAEKS scheme

The Chen et al. DS-PAEKS scheme [4] is described below:

DS-PAEKS.Setup(λ): The setup algorithm takes a security parameter λ as input, and outputs a common parameter $pp = (\mathbb{G}, p, g_1, g_2, g_3, H)$, where $\mathbb{G}$ is a DDH-hard group with prime order p , $g_1, g_2, g_3 \in \mathbb{G}$ are distinct generators, and $H : \{0, 1\} \rightarrow \mathbb{Z}_p$ is a collision-resistant hash function.

DS-PAEKS.KG_R(pp): Choose $d \xleftarrow{\$} \mathbb{Z}_p$. Output $\text{pk}_R = g_3^d$ and $\text{sk}_R = d$.

DS-PAEKS.KG_S(pp): Choose $c \xleftarrow{\$} \mathbb{Z}_p$. Output $\text{pk}_S = g_3^c$ and $\text{sk}_S = c$.

DS-PAEKS.KG_{AS}(pp): Choose $a \xleftarrow{\$} \mathbb{Z}_p$. Output $\text{pk}_{AS} = g_1^a$ and $\text{sk}_{AS} = a$.

DS-PAEKS.KG_{TS}(pp): Choose $b \xleftarrow{\$} \mathbb{Z}_p$. Output $\text{pk}_{TS} = g_2^b$ and $\text{sk}_{TS} = b$.

DS-PAEKS.Enc($\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{AS}, \text{pk}_{TS}, kw$): Choose $r_1 \xleftarrow{\$} \mathbb{Z}_p$. Compute $C_1 = g_1^{r_1}$, $C_2 = g_2^{r_1}$, and $C_3 = \text{pk}_{AS}^{r_1} \text{pk}_{TS}^{r_1} (\text{pk}_R^{\text{sk}_S})^{H(kw)}$ and output $\text{ct}_{\text{DS-PAEKS}} = (C_1, C_2, C_3)$. Here, $C_3 = (g_1^a)^{r_1} (g_2^b)^{r_1} (g_3^{cd})^{H(kw)}$ holds.

DS-PAEKS.Trapdoor($\text{pk}_R, \text{pk}_S, \text{sk}_R, \text{pk}_{AS}, \text{pk}_{TS}, kw'$): Choose $r_2 \xleftarrow{\$} \mathbb{Z}_p$. Compute $T_1 = g_1^{r_2}$, $T_2 = g_2^{r_2}$, and $T_3 = \text{pk}_{AS}^{r_2} \text{pk}_{TS}^{r_2} / (\text{pk}_S^{\text{sk}_R})^{H(kw')}$, and output $\text{td}_{\text{DS-PAEKS}} = (T_1, T_2, T_3)$. Here, $T_3 = (g_1^a)^{r_2} (g_2^b)^{r_2} / (g_3^{cd})^{H(kw')}$ holds.

DS-PAEKS.Transition($\text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{sk}_{AS}, \text{ct}_{\text{DS-PAEKS}}, \text{td}_{\text{DS-PAEKS}}$): Parse $\text{sk}_{AS} = a$, $\text{ct}_{\text{DS-PAEKS}} = (C_1, C_2, C_3)$, and $\text{td}_{\text{DS-PAEKS}} = (T_1, T_2, T_3)$. Choose $r_3 \xleftarrow{\$} \mathbb{Z}_p$. Compute

$$\begin{aligned} ICT_1 &= \{(C_3 \cdot T_3) / (C_1 \cdot T_1)^a\}^{r_3} \\ &= \{(g_1^a)^{r_1+r_2} (g_2^b)^{r_1+r_2} (g_3^{cd})^{H(kw)-H(kw')} / (g_1^{r_1+r_2})^a\}^{r_3} \\ &= (g_2^b)^{r_3(r_1+r_2)} (g_3^{cd})^{r_3(H(kw)-H(kw'))} \end{aligned}$$

and $ICT_2 = (C_2 \cdot T_2)^{r_3} = g_2^{r_3(r_1+r_2)}$. Output $\text{int-ct}_{\text{DS-PAEKS}} = (ICT_1, ICT_2)$.

DS-PAEKS.Test($\text{pk}_R, \text{pk}_S, \text{pk}_{TS}, \text{sk}_{TS}, \text{int-ct}_{\text{DS-PAEKS}}$): Parse $\text{sk}_{TS} = b$ and $\text{int-ct}_{\text{DS-PAEKS}} = (ICT_1, ICT_2)$. Output 1 if $ICT_1 = ICT_2^b$ holds, and 0 otherwise.

Chen et al. claimed that $ICT_1 = (g_2^b)^{r_3(r_1+r_2)} (g_3^{cd})^{r_3(H(kw)-H(kw'))} = (g_2^b)^{r_3(r_1+r_2)} = (g_2^{r_3(r_1+r_2)})^b = ICT_2^b$ holds if $kw = kw'$. Due to the collision resistance of H , $H(kw) \neq H(kw')$ holds if $kw \neq kw'$. Thus, the DS-PAEKS.Test algorithm outputs 0 if $kw \neq kw'$.

Our attack is described here. The main problem is that the forms of ciphertext $\text{ct}_{\text{DS-PAEKS}} = (C_1, C_2, C_3)$ and trapdoor $\text{td}_{kw'}$ are almost the same, and an intermediate ciphertext $\text{int-ct}_{\text{DS-PAEKS}}$ can be constructed from two ciphertexts (and $\text{sk}_{AS} = a$) without using any trapdoor. Let $\text{ct}_{\text{DS-PAEKS}}^* = (C_1^*, C_2^*, C_3^*)$ be the challenge ciphertext where $C_1^* = g_1^{r_1^*}$, $C_2^* = g_2^{r_1^*}$, and $C_3^* = \text{pk}_{AS}^{r_1^*} \text{pk}_{TS}^{r_1^*} (\text{pk}_R^{\text{sk}_S})^{H(kw_\beta^*)}$ and $\beta \in \{0, 1\}$. The IND-AS-CKA adversary that has $\text{sk}_{AS} = a$ issues kw_0^* to the encryption oracle $\mathcal{O}_C$, and obtains $\text{ct}_{\text{DS-PAEKS}} = (C_1, C_2, C_3)$ where $C_1 = g_1^{r_1}$, $C_2 = g_2^{r_1}$, and $C_3 = \text{pk}_{AS}^{r_1} \text{pk}_{TS}^{r_1} (\text{pk}_R^{\text{sk}_S})^{H(kw_0^*)}$. The adversary chooses $r_3 \xleftarrow{\$} \mathbb{Z}_p$ and computes an intermediate ciphertext $ICT_1 = \{(C_3^*/C_3) / (C_1^*/C_1)^a\}^{r_3}$ and $ICT_2 = (C_2^*/C_2)^{r_3}$. Here,

$$\begin{aligned} ICT_1 &= \{(g_1^a)^{r_1^*-r_1} (g_2^b)^{r_2^*-r_2} (g_3^{cd})^{H(kw_\beta^*)-H(kw_0^*)} / (g_1^{r_1^*-r_1})^a\}^{r_3} \\ &= (g_2^b)^{r_3(r_2^*-r_2)} (g_3^{cd})^{r_3(H(kw_\beta^*)-H(kw_0^*))} \\ ICT_2 &= g_2^{r_3(r_2^*-r_2)} \end{aligned}$$

hold. The adversary sends $\text{int-ct}_{\text{DS-PAEKS}} = (ICT_1, ICT_2)$ to the test oracle $\mathcal{O}_{\text{Test}}$. If $\beta = 0$, then $ICT_1 = ICT_2^b$ holds and thus, the oracle outputs 1, and 0 otherwise. Thus, the adversary wins.

We give attacks against the Chen et al. DS-PEKS scheme [5] and the Tso et al. semi-generic construction [18] in the Appendix.

5 Proposed Generic construction

Technical Overview: The proposed generic construction employs Chen’s idea [6], i.e., when a server has a public key, and the test algorithm takes the secret key of the server as input, then it is sufficient to encrypt a trapdoor by the public key to hide information about the keywords associated with the trapdoor. In our construction, the assistant and test servers manage public keys of PKE, respectively.

- A sender re-encrypts a PAEKS ciphertext ct_{PAEKS} using pk_{AS} , and sends it to the assistant server as a DS-PAEKS ciphertext $\text{ct}_{\text{DS-PAEKS}}$.
- A receiver encrypts a PAEKS trapdoor $\text{td}'_{kw'}$ using pk_{TS} , and sends it to the assistant server as a DS-PAEKS trapdoor $\text{td}_{\text{DS-PAEKS}}$.
- The assistant server then decrypts the DS-PAEKS ciphertext $\text{ct}_{\text{DS-PAEKS}}$ using sk_{AS} , and sets the PAEKS ciphertext ct_{PAEKS} and the DS-PAEKS trapdoor $\text{td}_{\text{DS-PAEKS}}$ as the intermediate ciphertext.
- The test server decrypts the DS-PAEKS trapdoor $\text{td}_{\text{DS-PAEKS}}$ using sk_{TS} , and obtains the PAEKS trapdoor $\text{td}'_{kw'}$. The test server then runs the test algorithm of the underlying PAEKS scheme for $(\text{ct}_{\text{PAEKS}}, \text{td}'_{kw'})$.

The construction is basically secure because no information about keywords is leaked from PAEKS ciphertexts and PAEKS trapdoors due to the security of the underlying PAEKS scheme. Moreover, no single server can run the PAEKS test algorithm because either a PAEKS ciphertext or a PAEKS trapdoor is encrypted using the public key of the other server. We also introduce two signature schemes, where a sender and a receiver sign a PAEKS ciphertext and a PAEKS trapdoor, respectively, before the encryption to exclude the case of an adversary producing a PKE ciphertext of self-made PAEKS ciphertexts/trapdoors for the challenge keyword.

We construct a DS-PAEKS scheme $\text{DS-PAEKS} = (\text{DS-PAEKS.Setup}, \text{DS-PAEKS.KG}_R, \text{DS-PAEKS.KG}_S, \text{DS-PAEKS.KG}_{\text{AS}}, \text{DS-PAEKS.KG}_{\text{TS}}, \text{DS-PAEKS.Enc}, \text{DS-PAEKS.Trapdoor}, \text{DS-PAEKS.Transition}, \text{DS-PAEKS.Test})$ as follows. Let $\text{PAEKS} = (\text{PAEKS.Setup}, \text{PAEKS.KG}_R, \text{PAEKS.KG}_S, \text{PAEKS.Enc}, \text{PAEKS.Trapdoor}, \text{PAEKS.Test})$ be a PAEKS scheme, $\text{PKE} = (\text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a PKE scheme, and $\text{Sig} = (\text{Sig.KeyGen}, \text{Sign}, \text{Verify})$ be a signature scheme.

Generic Construction of DS-PAEKS

DS-PAEKS.Setup(λ): Run $\text{pp} \leftarrow \text{PAEKS.Setup}(1^\lambda)$ and output pp . We assume that pp contains the security parameter λ .

DS-PAEKS.KG_R(pp): Run $(\text{pk}'_R, \text{sk}'_R) \leftarrow \text{PAEKS.KG}_R(\text{pp})$ and $(\text{vk}_R, \text{sigk}_R) \leftarrow \text{Sig.KeyGen}(1^\lambda)$. Output $\text{pk}_R = (\text{pk}'_R, \text{vk}_R)$ and $\text{sk}_R = (\text{sk}'_R, \text{sigk}_R)$.

DS-PAEKS.KG_S(pp): Run $(\text{pk}'_S, \text{sk}'_S) \leftarrow \text{PAEKS.KG}_S(\text{pp})$ and $(\text{vk}_S, \text{sigk}_S) \leftarrow \text{Sig.KeyGen}(1^\lambda)$. Output $\text{pk}_S = (\text{pk}'_S, \text{vk}_S)$ and $\text{sk}_S = (\text{sk}'_S, \text{sigk}_S)$.

DS-PAEKS.KG_{AS}(pp): Run $(\text{PK}, \text{DK}) \leftarrow \text{PKE.KeyGen}(1^\lambda)$ and output $\text{pk}_{\text{AS}} = \text{PK}$ and $\text{sk}_{\text{AS}} = \text{DK}$.

DS-PAEKS.KG_{TS}(pp): Run $(\text{PK}', \text{DK}') \leftarrow \text{PKE.KeyGen}(1^\lambda)$ and output $\text{pk}_{\text{TS}} = \text{PK}'$ and $\text{sk}_{\text{TS}} = \text{DK}'$.

DS-PAEKS.Enc($\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{\text{AS}}, \text{pk}_{\text{TS}}, kw$): Parse $\text{pk}_R = (\text{pk}'_R, \text{vk}_R)$, $\text{pk}_S = (\text{pk}'_S, \text{vk}_S)$, and $\text{sk}_S = (\text{sk}'_S, \text{sigk}_S)$. Run $\text{ct}_{\text{PAEKS}} \leftarrow \text{PAEKS.Enc}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_S, kw)$, $\sigma \leftarrow \text{Sign}(\text{sigk}_S, \text{ct}_{\text{PAEKS}})$, and $C \leftarrow \text{PKE.Enc}(\text{pk}_{\text{AS}}, \sigma || \text{ct}_{\text{PAEKS}})$. Output $\text{ct}_{\text{DS-PAEKS}} = C$.

DS-PAEKS.Trapdoor($\text{pk}_R, \text{pk}_S, \text{sk}_R, \text{pk}_{AS}, \text{pk}_{TS}, kw'$): Parse $\text{pk}_R = (\text{pk}'_R, \text{vk}_R)$, $\text{sk}_R = (\text{sk}'_R, \text{sigk}_R)$, and $\text{pk}_S = (\text{pk}'_S, \text{vk}_S)$. Run $\text{td}'_{kw'} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_R, kw')$, $\sigma' \leftarrow \text{Sign}(\text{sigk}_R, \text{td}'_{kw'})$, and $C' \leftarrow \text{PKE.Enc}(\text{pk}_{TS}, \sigma' || \text{td}'_{kw'})$. Output $\text{td}_{\text{DS-PAEKS}} = C'$.

DS-PAEKS.Transition($\text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{sk}_{AS}, \text{ct}_{\text{DS-PAEKS}}, \text{td}_{\text{DS-PAEKS}}$): Parse $\text{pk}_S = (\text{pk}'_S, \text{vk}_S)$ and $\text{ct}_{\text{DS-PAEKS}} = C$. Run $\sigma || \text{ct}_{\text{PAEKS}} \leftarrow \text{PKE.Dec}(\text{sk}_{AS}, C)$. Output $\perp$ if $\text{Verify}(\text{vk}_S, \text{ct}_{\text{PAEKS}}, \sigma) = 0$. Otherwise, output $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ct}_{\text{PAEKS}}, \sigma, \text{td}_{\text{DS-PAEKS}})$.

DS-PAEKS.Test($\text{pk}_R, \text{pk}_S, \text{pk}_{TS}, \text{sk}_{TS}, \text{int-ct}_{\text{DS-PAEKS}}$): Parse $\text{pk}_R = (\text{pk}'_R, \text{vk}_R)$, $\text{pk}_S = (\text{pk}'_S, \text{vk}_S)$, and $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ct}_{\text{PAEKS}}, \sigma, \text{td}_{\text{DS-PAEKS}})$ where $\text{td}_{\text{DS-PAEKS}} = C'$. Output 0 if $\text{Verify}(\text{vk}_S, \text{ct}_{\text{PAEKS}}, \sigma) = 0$. Otherwise, run $\sigma' || \text{td}'_{kw'} \leftarrow \text{PKE.Dec}(\text{sk}_{TS}, C')$. Output 0 if $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 0$. Otherwise, output the result of $\text{PAEKS.Test}(\text{ct}_{\text{PAEKS}}, \text{td}'_{kw'})$.

The proposed construction is correct if the underlying PAEKS, PKE, and signature schemes are correct. Moreover, since the DS-PAEKS.Test algorithm returns the result of the PAEKS.Test algorithm, the proposed construction is computationally consistent if the underlying PAEKS scheme is computationally consistent.

6 Security Analysis

Theorem 1. *The proposed construction is IND-AS-CKA secure if PAEKS is IND-CKA secure, PKE is IND-CCA secure, and Sig is sEUF-CMA secure.*

Basically, the IND-AS-CKA security is reduced to the IND-CKA security of PAEKS. However, we must consider two main cases: (1) how to simulate $\mathcal{O}_{\text{Trap}}$ for $kw \in \{kw_0^*, kw_1^*\}$ because $\mathcal{O}_{\text{Trap}}$ of the underlying PAEKS scheme has the restriction that $kw \notin \{kw_0^*, kw_1^*\}$, and (2) how to prevent any modification of trapdoors of the challenge keyword because if an adversary issues $\text{int-ct}_{\text{DS-PAEKS}}$ to $\mathcal{O}_{\text{Test}}$, where $\text{int-ct}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Transition}(\text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{sk}_{AS}, \text{ct}_{\text{DS-PAEKS}}^*, \text{td}_{\text{DS-PAEKS}}) \wedge \text{td}_{\text{DS-PAEKS}} \notin \text{TSet}$, and $\text{td}_{\text{DS-PAEKS}}$ is a DS-PAEKS trapdoor for either kw_0^* or kw_1^* , then the adversary trivially wins. We handled the first issue by employing the IND-CPA security of the underlying PKE scheme. PAEKS trapdoors are now encrypted by the public key of the test server. Thus, the PKE ciphertext of the PAEKS trapdoor for $kw \in \{kw_0^*, kw_1^*\}$ can be replaced with a PKE ciphertext of 0^ℓ (where ℓ is the bit size of the PAEKS trapdoor) due to the IND-CPA security. Then, the simulator does not have to issue a trapdoor query to the underlying PAEKS scheme. More precisely, we require that the underlying PKE scheme is IND-CCA secure to simulate $\mathcal{O}_{\text{Test}}$ that internally runs the decryption algorithm of PKE. We handled the second issue by employing the sEUF-CMA security of the underlying signature scheme. That is, a PAEKS trapdoor is signed before encryption to prevent any PAEKS trapdoor modification.

Proof. The proof uses a sequence of games. Let E_i be the event in which $\mathcal{A}$ outputs $\beta' = \beta$ in Game i .

Game 0: This game corresponds to the real game. By definition, $\text{Adv}_{\text{DS-PAEKS}, \mathcal{A}}^{\text{IND-AS-CKA}}(\lambda) = |\Pr[E_0] - 1/2|$.

Game 1: This game is the same as Game 0, except that the response of the $\mathcal{O}_{\text{Test}}$ oracle is changed as follows. Let $\mathcal{A}$ be an IND-AS-CKA adversary. Assume that $\mathcal{A}$ issues $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ct}_{\text{PAEKS}}, \sigma, \text{td}_{\text{DS-PAEKS}})$ to $\mathcal{O}_{\text{Test}}$. Run $\sigma' || \text{td}'_{kw'} \leftarrow \text{PKE.Dec}(\text{sk}_{TS}, \text{td}_{\text{DS-PAEKS}})$. If $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 1$ and $(\text{td}'_{kw'}, \sigma')$ was not generated in the $\mathcal{O}_{\text{Trap}}$ oracle, then the challenger aborts. If the challenger does not abort, then Game 1 is identical to Game 0. Thus, $|\Pr[E_0] - \Pr[E_1]| \leq \Pr[\text{abort}]$ where abort is the event that the challenger aborts.

Lemma 1. *There exists an algorithm $\mathcal{B}$ such that $\Pr[\text{abort}] \leq \text{Adv}_{\text{Sig}, \mathcal{B}}^{\text{sEUF-CMA}}(\lambda)$.*

Proof. Let $\mathcal{A}$ be the adversary of IND-AS-CKA and $\mathcal{C}$ be the challenger of the signature scheme. We construct an algorithm $\mathcal{B}$ that breaks the sEUF-CMA security as follows. First, $\mathcal{B}$ runs $\text{pp} \leftarrow \text{PAEKS.Setup}(1^\lambda)$, $(\text{pk}'_R, \text{sk}'_R) \leftarrow \text{PAEKS.KG}_R(\text{pp})$, $(\text{pk}'_S, \text{sk}'_S) \leftarrow \text{PAEKS.KG}_S(\text{pp})$, $(\text{vk}_S, \text{sigk}_S) \leftarrow \text{Sig.KeyGen}(1^\lambda)$, $(\text{PK}, \text{DK}) \leftarrow \text{PKE.KeyGen}(1^\lambda)$, and $(\text{PK}', \text{DK}') \leftarrow \text{PKE.KeyGen}(1^\lambda)$. $\mathcal{C}$ runs $(\text{vk}, \text{sigk}) \leftarrow \text{Sig.KeyGen}(1^\lambda)$, and sends vk to $\mathcal{B}$. $\mathcal{B}$ sets $\text{pk}_R = (\text{pk}'_R, \text{vk})$, $\text{sk}_R = (\text{sk}'_R, -)$, $\text{pk}_S = (\text{pk}'_S, \text{vk}_S)$, $\text{sk}_S = (\text{sk}'_S, \text{sigk}_S)$, $\text{pk}_{AS} = \text{PK}$, $\text{sk}_{AS} = \text{DK}$, $\text{pk}_{TS} = \text{PK}'$, and $\text{sk}_{TS} = \text{DK}'$, and sends $(\text{pp}, \text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{sk}_{AS}, \text{pk}_{TS})$ to $\mathcal{A}$. $\mathcal{B}$ sets $\text{TSet} := \emptyset$ and $\text{SSet} := \emptyset$.

- For $\mathcal{O}_C$, $\mathcal{B}$ can respond to any query because $\mathcal{B}$ has sk_S .
- For $\mathcal{O}_{\text{Trap}}$, $\mathcal{B}$ can respond to a query kw' from $\mathcal{A}$ as follows. $\mathcal{B}$ runs $\text{td}'_{kw'} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_R, kw')$ and sends $\text{td}'_{kw'}$ to $\mathcal{C}$ as a signing query. $\mathcal{C}$ returns $\sigma' \leftarrow \text{Sign}(\text{sigk}, \text{td}'_{kw'})$ to $\mathcal{B}$. $\mathcal{B}$ computes $C' \leftarrow \text{PKE.Enc}(\text{pk}_{TS}, \sigma' || \text{td}'_{kw'})$ and returns $\text{td}_{\text{DS-PAEKS}} = C'$ to $\mathcal{A}$. Moreover, $\mathcal{B}$ stores $(\text{td}'_{kw'}, \sigma')$ on SSet .
- For $\mathcal{O}_{\text{Test}}$, $\mathcal{B}$ can respond to a query $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ct}_{\text{PAEKS}}, \sigma, \text{td}_{\text{DS-PAEKS}})$ from $\mathcal{A}$ as follows. $\mathcal{B}$ returns 0 if $\text{Verify}(\text{vk}_S, \text{ct}_{\text{PAEKS}}, \sigma) = 0$. Otherwise, $\mathcal{B}$ runs $\sigma' || \text{td}'_{kw'} \leftarrow \text{PKE.Dec}(\text{sk}_{TS}, \text{td}_{\text{DS-PAEKS}})$. $\mathcal{B}$ returns 0 if $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 0$. From now on, we assume that $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 1$. If $(\text{td}'_{kw'}, \sigma')$ is generated in the $\mathcal{O}_{\text{Trap}}$ oracle, then $\mathcal{B}$ returns the result of $\text{PAEKS.Test}(\text{ct}_{\text{PAEKS}}, \text{td}'_{kw'})$. If $(\text{td}'_{kw'}, \sigma')$ was not generated in the $\mathcal{O}_{\text{Trap}}$ oracle, i.e., $(\text{td}'_{kw'}, \sigma') \notin \text{SSet}$, then σ' is not a response from $\mathcal{C}$. Thus, $\mathcal{B}$ outputs $(\text{td}'_{kw'}, \sigma')$ as a forged message and signature pair, and breaks the sEUF-CMA security of the signature scheme.

In the challenge phase, $\mathcal{A}$ declares (kw_0^*, kw_1^*) . $\mathcal{B}$ chooses $\beta \xleftarrow{\$} \{0, 1\}$, generates $\text{ct}_{\text{DS-PAEKS}}^* \leftarrow \text{DS-PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{AS}, \text{pk}_{TS}, kw_\beta^*)$, and returns $\text{ct}_{\text{DS-PAEKS}}^*$ to $\mathcal{A}$.

$\mathcal{B}$ simulates $\mathcal{O}_C$, $\mathcal{O}_{\text{Trap}}$, and $\mathcal{O}_{\text{Test}}$ as in the previous phase, except that if $\mathcal{A}$ sends $kw \in \{kw_0^*, kw_1^*\}$ to $\mathcal{O}_{\text{Trap}}$, then $\mathcal{B}$ updates $\text{TSet} = \text{TSet} \cup \{\text{td}_{\text{DS-PAEKS}}\}$ where $\text{td}_{\text{DS-PAEKS}}$ is the response of the $\mathcal{O}_{\text{Trap}}$ oracle. If $\mathcal{A}$ does not issue a test query $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ct}_{\text{PAEKS}}, \sigma, \text{td}_{\text{DS-PAEKS}})$ where, for $\sigma' || \text{td}'_{kw'} \leftarrow \text{PKE.Dec}(\text{sk}_{TS}, \text{td}_{\text{DS-PAEKS}})$, $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 1$ and $(\text{td}'_{kw'}, \sigma')$ was not generated in the $\mathcal{O}_{\text{Trap}}$ oracle, then $\mathcal{B}$ simulates Game 0, and Game 1 otherwise. $\square$

Game 2.k ($1 \leq k \leq q_{\text{Trap}}$): Let q_{Trap} be the number of trapdoor queries issued by $\mathcal{A}$ and Game 2.0 be the same as Game 1. Game 2.k is the same as Game 2.k-1, except that the response of the k -th $\mathcal{O}_{\text{Trap}}$ query is changed as follows. Let $\mathcal{A}$ issues kw to $\mathcal{O}_{\text{Trap}}$ as the k -th query. Let ℓ be the bit size of the PAEKS trapdoor. Set $\text{td}'_{kw} = 0^{|\ell|}$ and run $\sigma' \leftarrow \text{Sign}(\text{sigk}_R, \text{td}'_{kw})$, and $C' \leftarrow \text{PKE.Enc}(\text{pk}_{TS}, \sigma' || \text{td}'_{kw})$. Output $\text{td}_{\text{DS-PAEKS}} = C'$.

Lemma 2. *For each $k \in [1, q_{\text{Trap}}]$, Game 2.k is indistinguishable from Game 2.k-1 if the underlying PKE scheme is IND-CCA secure. Precisely, there exists an algorithm $\mathcal{B}$ such that $|\Pr[E_{2.k-1}] - \Pr[E_{2.k}]| \leq \text{Adv}_{\text{PKE}, \mathcal{B}}^{\text{IND-CCA}}(\lambda)$.*

Proof. Let $\mathcal{A}$ be the adversary of IND-AS-CKA and $\mathcal{C}$ be the challenger of the PKE scheme. We construct an algorithm $\mathcal{B}$ that breaks the IND-CCA security as follows. $\mathcal{C}$ runs $(\text{PK}', \text{DK}') \leftarrow \text{PKE.KeyGen}(1^\lambda)$ and sends PK' to $\mathcal{B}$. $\mathcal{B}$ runs $\text{pp} \leftarrow \text{PAEKS.Setup}(1^\lambda)$, $(\text{pk}'_R, \text{sk}'_R) \leftarrow \text{PAEKS.KG}_R(\text{pp})$, $(\text{vk}_R, \text{sigk}_R) \leftarrow \text{Sig.KeyGen}(1^\lambda)$, $(\text{pk}'_S, \text{sk}'_S) \leftarrow \text{PAEKS.KG}_S(\text{pp})$, $(\text{vk}_S, \text{sigk}_S) \leftarrow \text{Sig.KeyGen}(1^\lambda)$, and $(\text{PK}, \text{DK}) \leftarrow \text{PKE.KeyGen}(1^\lambda)$. $\mathcal{B}$ sets $\text{pk}_{TS} = (\text{pk}'_R, \text{vk})$, $\text{sk}_R = (\text{sk}'_R, \text{sigk}_R)$, $\text{pk}_S = (\text{pk}'_S, \text{vk}_S)$, $\text{sk}_S = (\text{sk}'_S, \text{sigk}_S)$, $\text{pk}_{AS} = \text{PK}$, $\text{sk}_{AS} = \text{DK}$, $\text{pk}_{TS} = \text{PK}'$, and $\text{sk}_{TS} = -$, and sends $(\text{pp}, \text{pk}_R, \text{pk}_S, \text{pk}_{AS}, \text{sk}_{AS}, \text{pk}_{TS})$ to $\mathcal{A}$. $\mathcal{B}$ sets $\text{TSet} := \emptyset$, $\text{SSet} := \emptyset$, and $\text{CSet} := \emptyset$.

- For $\mathcal{O}_C$, $\mathcal{B}$ can respond to any query because $\mathcal{B}$ has sk_S .
- For $\mathcal{O}_{\text{Trap}}$, $\mathcal{B}$ can respond to a query kw' from $\mathcal{A}$ as follows.
 - From 1 to $k - 1$ -th queries, $\mathcal{B}$ sets $\text{td}'_{kw'} = 0^{|\ell|}$, computes $\sigma' \leftarrow \text{Sign}(\text{sigk}_R, \text{td}'_{kw'})$ and $C' \leftarrow \text{PKE.Enc}(\text{pk}_{TS}, \sigma' || \text{td}'_{kw'})$, and returns $\text{td}_{\text{DS-PAEKS}} = C'$ to $\mathcal{A}$. Moreover, $\mathcal{B}$ stores $(kw', \text{td}'_{kw'}, \sigma')$ on SSet.
 - From $k + 1$ to q_{Trap} queries, $\mathcal{B}$ runs $\text{td}'_{kw'} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_R, kw')$, $\sigma' \leftarrow \text{Sign}(\text{sigk}_R, \text{td}'_{kw'})$, and $C' \leftarrow \text{PKE.Enc}(\text{pk}_{TS}, \sigma' || \text{td}'_{kw'})$, and returns $\text{td}_{\text{DS-PAEKS}} = C'$ to $\mathcal{A}$. Moreover, $\mathcal{B}$ stores $(kw', \text{td}'_{kw'}, \sigma')$ on SSet.
 - For the k -th query, $\mathcal{B}$ runs $\text{td}'_{kw'} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_R, kw')$ and computes $\sigma' \leftarrow \text{Sign}(\text{sigk}_R, \text{td}'_{kw'})$ and $\sigma'' \leftarrow \text{Sign}(\text{sigk}_R, 0^{|\ell|})$. $\mathcal{B}$ sets $(\sigma' || \text{td}'_{kw'}, \sigma'' || 0^{|\ell|})$ as the challenge plaintexts, and sends $(\sigma' || \text{td}'_{kw'}, \sigma'' || 0^{|\ell|})$ to $\mathcal{C}$. $\mathcal{C}$ returns the challenge ciphertext C^* . $\mathcal{B}$ returns $\text{td}_{\text{DS-PAEKS}} = C^*$ to $\mathcal{A}$. Moreover, $\mathcal{B}$ stores (kw', C^*) on CSet.
- For $\mathcal{O}_{\text{Test}}$, $\mathcal{B}$ can respond to a query $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ctPAEKS}, \sigma, \text{td}_{\text{DS-PAEKS}})$ from $\mathcal{A}$ as follows. $\mathcal{B}$ returns 0 if $\text{Verify}(\text{vk}_S, \text{ctPAEKS}, \sigma) = 0$. If $\text{td}_{\text{DS-PAEKS}} = C^*$, then $\mathcal{B}$ knows the keyword associated to $\text{td}_{\text{DS-PAEKS}}$ because (kw', C^*) is stored on CSet. $\mathcal{B}$ runs $\text{td}'_{kw'} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_R, kw')$, and returns the result of $\text{PAEKS.Test}(\text{ctPAEKS}, \text{td}'_{kw'})$. If $\text{td}_{\text{DS-PAEKS}} \neq C^*$, then $\mathcal{B}$ sends $\text{td}_{\text{DS-PAEKS}}$ to $\mathcal{C}$ as a decryption query. If $\mathcal{C}$ returns $\perp$, then $\mathcal{B}$ returns 0 to $\mathcal{A}$. Otherwise, let $\sigma' || \text{td}'_{kw'}$ be the response from $\mathcal{C}$. $\mathcal{B}$ returns 0 if $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 0$. Otherwise, $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 1$. Due to the modification of Game 1, $(\text{td}'_{kw'}, \sigma')$ was generated in the $\mathcal{O}_{\text{Trap}}$ oracle. Thus, $(kw', \text{td}'_{kw'}, \sigma') \in \text{SSet}$. If $\text{td}'_{kw'} = 0^{|\ell|}$, then $\mathcal{B}$ runs $\text{td}'_{kw'} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_R, kw')$. $\mathcal{B}$ returns the result of $\text{PAEKS.Test}(\text{ctPAEKS}, \text{td}'_{kw'})$.

In the challenge phase, $\mathcal{A}$ declares (kw_0^*, kw_1^*) . $\mathcal{B}$ chooses $\beta \xleftarrow{\$} \{0, 1\}$, generates $\text{ct}_{\text{DS-PAEKS}}^* \leftarrow \text{DS-PAEKS.Enc}(\text{pk}_R, \text{pk}_S, \text{sk}_S, \text{pk}_{AS}, \text{pk}_{TS}, kw_\beta^*)$, and returns $\text{ct}_{\text{DS-PAEKS}}^*$ to $\mathcal{A}$.

$\mathcal{B}$ simulates $\mathcal{O}_C$, $\mathcal{O}_{\text{Trap}}$, and $\mathcal{O}_{\text{Test}}$ as in the previous phase, except that if $\mathcal{A}$ sends $kw \in \{kw_0^*, kw_1^*\}$ to $\mathcal{O}_{\text{Trap}}$, then $\mathcal{B}$ updates $\text{TSet} = \text{TSet} \cup \{\text{td}_{\text{DS-PAEKS}}\}$ where $\text{td}_{\text{DS-PAEKS}}$ is the response of the $\mathcal{O}_{\text{Trap}}$ oracle. If the challenge ciphertext C^* is an encryption of $\sigma' || \text{td}'_{kw}$, then $\mathcal{B}$ simulates Game 2. $k - 1$, and if C^* is an encryption of $\sigma'' || 0^{|\ell|}$, then $\mathcal{B}$ simulates Game 2. k . Thus, $|\Pr[E_{2.k-1}] - \Pr[E_{2.k}]| \leq \text{Adv}_{\text{PKE}, \mathcal{B}}^{\text{IND-CCA}}(\lambda)$ holds. $\square$

Game 3: This game is the same as Game 2. q_{Trap} , except that the response of $\mathcal{O}_{\text{Test}}$ is changed as follows. If $\mathcal{A}$ issues $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ctPAEKS}, \sigma, \text{td}_{\text{DS-PAEKS}})$ such that (1) $\text{Verify}(\text{vk}_S, \text{ctPAEKS}, \sigma) = 1$, (2) for $\sigma' || \text{td}'_{kw} \leftarrow \text{PKE.Dec}(\text{sk}_{TS}, \text{td}_{\text{DS-PAEKS}})$, $(\text{td}'_{kw}, \sigma')$ was generated in the $\mathcal{O}_{\text{Trap}}$ oracle for a query kw (and thus $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 1$ and $(kw, \text{td}'_{kw'}, \sigma') \in \text{SSet}$), (3) $kw \in \{kw_0^*, kw_1^*\}$, and (4) $\text{td}_{\text{DS-PAEKS}} \notin \text{TSet}$, then the challenger aborts. If challenger does not abort, then Game 3 is identical to Game 2. q_{Trap} . Thus, $|\Pr[E_{2.q_{\text{Trap}}}] - \Pr[E_3]| \leq \Pr[\text{abort}]$ where abort is the event when the challenger aborts.

Lemma 3. *There exists an algorithm $\mathcal{B}$ such that $\Pr[\text{abort}] \leq q_{\text{Trap}} \cdot \text{Adv}_{\text{PKE}, \mathcal{B}}^{\text{IND-CCA}}(\lambda)$.*

Proof. Due to the modification in Game 1, $(kw, \text{td}'_{kw}, \sigma') \in \text{SSet}$. Thus, $kw \in \{kw_0^*, kw_1^*\}$ and $\text{td}_{\text{DS-PAEKS}} \notin \text{TSet}$ mean that a PKE ciphertext $\text{PKE.Enc}(\text{pk}_{TS}, \sigma' || \text{td}'_{kw})$ is re-randomized by $\mathcal{A}$ that contradicts the IND-CCA security. Let $\mathcal{A}$ be the adversary of IND-AS-CCA and $\mathcal{C}$ be the challenger of the PKE scheme.

We construct an algorithm $\mathcal{B}$ that breaks the IND-CCA security as follows. $\mathcal{C}$ runs $(PK', DK') \leftarrow \text{PKE.KeyGen}(1^\lambda)$ and sends PK' to $\mathcal{B}$. $\mathcal{B}$ runs $pp \leftarrow \text{PAEKS.Setup}(1^\lambda)$, $(pk'_R, sk'_R) \leftarrow \text{PAEKS.KG}_R(pp)$, $(vk_R, sigk_R) \leftarrow \text{Sig.KeyGen}(1^\lambda)$, $(pk'_S, sk'_S) \leftarrow \text{PAEKS.KG}_S(pp)$, $(vk_S, sigk_S) \leftarrow \text{Sig.KeyGen}(1^\lambda)$, and $(PK, DK) \leftarrow \text{PKE.KeyGen}(1^\lambda)$. $\mathcal{B}$ sets $pk_{TS} = (pk'_R, vk)$, $sk_R = (sk'_R, sigk_R)$, $pk_S = (pk'_S, vk_S)$, $sk_S = (sk'_S, sigk_S)$, $pk_{AS} = PK$, $sk_{AS} = DK$, $pk_{TS} = PK'$, and $sk_{TS} = -$, and sends $(pp, pk_R, pk_S, pk_{AS}, sk_{AS}, pk_{TS})$ to $\mathcal{A}$. $\mathcal{B}$ sets $TSet := \emptyset$ and $SSet := \emptyset$. We assume that (td'_{kw}, σ') (appeared in the condition (2)) was generated in the $\mathcal{O}_{\text{Trap}}$ oracle for the k^* -th trapdoor query. $\mathcal{B}$ guesses k^* . From now on, we assume that the guessing of k^* is correct (with the probability at least $1/q_{\text{Trap}}$).

In the challenge phase, $\mathcal{A}$ declares (kw_0^*, kw_1^*) . $\mathcal{B}$ chooses $\beta \xleftarrow{\$} \{0, 1\}$, generates $ct_{\text{DS-PAEKS}}^* \leftarrow \text{DS-PAEKS.Enc}(pk_R, pk_S, sk_S, pk_{AS}, pk_{TS}, kw_\beta^*)$, and returns $ct_{\text{DS-PAEKS}}^*$ to $\mathcal{A}$.

- For $\mathcal{O}_C$, $\mathcal{B}$ can respond to any query because $\mathcal{B}$ has sk_S .
- $\mathcal{B}$ simulates $\mathcal{O}_{\text{Trap}}$ oracle as in Game 2. q_{Trap} , except the k^* -th query. $\mathcal{B}$ simulates $\mathcal{O}_{\text{Trap}}$ for the k^* -th query as follows. We remark that the k^* -th query may be sent from $\mathcal{A}$ before the challenge phase. We also remark that, for a trapdoor query kw , $\mathcal{B}$ sets $td'_{kw} = 0^{|\ell|}$ regardless of the associated keyword due to the modification of previous games. $\mathcal{B}$ computes $\sigma'_0 \leftarrow \text{Sign}(sigk_R, 0^{|\ell|})$ and $\sigma'_1 \leftarrow \text{Sign}(sigk_R, 0^{|\ell|})$, and sets $(\sigma'_0 || 0^{|\ell|}, \sigma'_1 || 0^{|\ell|})$ as the challenge plaintexts. We remark that we have explicitly assumed that the Sign algorithm is probabilistic and thus $\sigma'_0 \neq \sigma'_1$ holds with overwhelming probability.² $\mathcal{C}$ returns the challenge ciphertext $C^* \leftarrow \text{PKE.Enc}(pk_{TS}, \sigma'_{\beta'}, 0^{|\ell|})$ where $\beta' \in \{0, 1\}$. $\mathcal{B}$ returns $td_{\text{DS-PAEKS}} = C^*$ to $\mathcal{A}$. $\mathcal{B}$ stores $(kw, 0^{|\ell|}, \sigma'_0, \sigma'_1)$ to $SSet$ (here, the 4-th entry is added to store two signatures).
- For $\mathcal{O}_{\text{Test}}$, $\mathcal{B}$ can respond to a query $\text{int-ct}_{\text{DS-PAEKS}} = (ct_{\text{PAEKS}}, \sigma, td_{\text{DS-PAEKS}})$ from $\mathcal{A}$ as follows. $\mathcal{B}$ returns 0 if $\text{Verify}(vk_S, ct_{\text{PAEKS}}, \sigma) = 0$. If $td_{\text{DS-PAEKS}} = C^*$, then $\mathcal{B}$ knows the keyword associated to $td_{kw'}$ because $(kw, 0^{|\ell|}, \sigma'_0, \sigma'_1)$ is stored on $SSet$. $\mathcal{B}$ runs $td'_{kw} \leftarrow \text{PAEKS.Trapdoor}(pk'_R, pk'_S, sk'_R, kw)$, and returns the result of $\text{PAEKS.Test}(ct_{\text{PAEKS}}, td'_{kw})$. If $td_{\text{DS-PAEKS}} \neq C^*$, then $\mathcal{B}$ sends $td_{kw'}$ to $\mathcal{C}$ as a decryption query. If $\mathcal{C}$ returns $\perp$, then $\mathcal{B}$ returns 0 to $\mathcal{A}$. Otherwise, let $\sigma' || td'_{kw'}$ be the response from $\mathcal{C}$. $\mathcal{B}$ returns 0 if $\text{Verify}(vk_R, td'_{kw'}, \sigma') = 0$. Otherwise, $\text{Verify}(vk_R, td'_{kw'}, \sigma') = 1$. Due to the modification of Game 1, $(td'_{kw'}, \sigma')$ was generated in the $\mathcal{O}_{\text{Trap}}$ oracle. If $(td'_{kw'}, \sigma')$ was generated in the $\mathcal{O}_{\text{Trap}}$ oracle for the k^* -th query, then $td'_{kw} = 0^{|\ell|}$, $\sigma' \in \{\sigma'_0, \sigma'_1\}$ where $(kw, 0^{|\ell|}, \sigma'_0, \sigma'_1) \in SSet$, $kw \in \{kw_0^*, kw_1^*\}$, and $td_{\text{DS-PAEKS}} \notin TSet$. $\mathcal{B}$ outputs 0 if $\sigma' = \sigma'_0$ and 1 if $\sigma' = \sigma'_1$ and breaks the IND-CCA security. If $(td'_{kw'}, \sigma')$ was generated in the $\mathcal{O}_{\text{Trap}}$ oracle for the k -th query where $k \neq k^*$, let $(kw', 0^{|\ell|}, \sigma') \in SSet$. $\mathcal{B}$ runs $td'_{kw'} \leftarrow \text{PAEKS.Trapdoor}(pk'_R, pk'_S, sk'_R, kw')$ and returns the result of $\text{PAEKS.Test}(ct_{\text{PAEKS}}, td'_{kw'})$. $\square$

Lemma 4. *There exists an algorithm $\mathcal{B}$ such that $|\Pr[E_3] - 1/2| \leq \text{Adv}_{\text{PAEKS}, \mathcal{B}}^{\text{IND-CKA}}(\lambda)$.*

Proof. Let $\mathcal{A}$ be the adversary of IND-AS-CKA and $\mathcal{C}$ be the challenger of the PAEKS scheme. We construct an algorithm $\mathcal{B}$ that breaks the IND-CKA security as follows. $\mathcal{C}$ runs $pp \leftarrow \text{PAEKS.Setup}(1^\lambda)$, $(pk'_R, sk'_R) \leftarrow \text{PAEKS.KG}_R(pp)$, and $(pk'_S, sk'_S) \leftarrow \text{PAEKS.KG}_S(pp)$, and sends (pp, pk'_R, pk'_S) to $\mathcal{B}$. $\mathcal{B}$ runs $(vk_R, sigk_R) \leftarrow \text{Sig.KeyGen}(1^\lambda)$, $(vk_S, sigk_S) \leftarrow \text{Sig.KeyGen}(1^\lambda)$, $(PK, DK) \leftarrow \text{PKE.KeyGen}(1^\lambda)$, and $(PK', DK') \leftarrow \text{PKE.KeyGen}(1^\lambda)$, and sets $pk_{TS} = (pk'_R, vk)$, $sk_R = (-, sigk_R)$, $pk_S = (pk'_S, vk_S)$, $sk_S = (-, sigk_S)$, $pk_{AS} = PK$, $sk_{AS} = DK$, $pk_{TS} = PK'$, and $sk_{TS} = DK'$, and sends $(pp, pk_R, pk_S, pk_{AS}, sk_{AS}, pk_{TS})$ to $\mathcal{A}$. $\mathcal{B}$ sets $SSet := \emptyset$.

²If the Sign algorithm is deterministic, e.g., the BLS signature scheme [3], then two challenge plaintexts are identical. This is the reason why we employ that a probabilistic signing algorithm in our construction.

- For $\mathcal{O}_C$, $\mathcal{B}$ can respond to a query kw from $\mathcal{A}$ as follows. $\mathcal{B}$ sends kw to $\mathcal{C}$ as an encryption query. Then, $\mathcal{C}$ generates $\text{ct}_{\text{PAEKS}} \leftarrow \text{PAEKS.Enc}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_S, kw)$, and sends ct_{PAEKS} to $\mathcal{B}$. $\mathcal{B}$ runs $\sigma \leftarrow \text{Sign}(\text{sigk}_S, \text{ct}_{\text{PAEKS}})$ and $C \leftarrow \text{PKE.Enc}(\text{pk}_{\text{AS}}, \sigma || \text{ct}_{\text{PAEKS}})$, and sends $\text{ct}_{\text{DS-PAEKS}} = C$ to $\mathcal{A}$.
- For $\mathcal{O}_{\text{Trap}}$, $\mathcal{B}$ can respond to a query kw from $\mathcal{A}$ as follows. $\mathcal{B}$ sets $\text{td}'_{kw} = 0^{|\ell|}$, computes $\sigma' \leftarrow \text{Sign}(\text{sigk}_R, \text{td}'_{kw})$ and $C' \leftarrow \text{PKE.Enc}(\text{pk}_{\text{TS}}, \sigma' || \text{td}'_{kw})$, and returns $\text{td}_{\text{DS-PAEKS}} = C'$ to $\mathcal{A}$. Moreover, $\mathcal{B}$ stores $(kw, 0^{|\ell|}, \sigma')$ to SSet .
- For $\mathcal{O}_{\text{Test}}$, $\mathcal{B}$ can respond to a query $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ct}_{\text{PAEKS}}, \sigma, \text{td}_{\text{DS-PAEKS}})$ from $\mathcal{A}$ as follows. $\mathcal{B}$ returns 0 if $\text{Verify}(\text{vk}_S, \text{ct}_{\text{PAEKS}}, \sigma) = 0$. Otherwise, $\mathcal{B}$ runs $\sigma' || \text{td}'_{kw'} \leftarrow \text{PKE.Dec}(\text{sk}_{\text{TS}}, \text{td}_{\text{DS-PAEKS}})$. $\mathcal{B}$ returns 0 if $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 0$. Otherwise, when $\text{Verify}(\text{vk}_R, \text{td}'_{kw'}, \sigma') = 1$, σ' has been stored such that $(kw, 0^{|\ell|}, \sigma') \in \text{SSet}$ due to the modification of Game 1. Moreover, $kw \notin \{kw_0^*, kw_1^*\}$ due to the modification of Game 3. Thus, regardless of whether $\mathcal{A}$ has declared (kw_0^*, kw_1^*) or not, $\mathcal{B}$ sends kw to $\mathcal{C}$ as a trapdoor query. $\mathcal{C}$ runs $\text{td}'_{kw} \leftarrow \text{PAEKS.Trapdoor}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_R, kw)$ and sends td'_{kw} to $\mathcal{B}$. $\mathcal{B}$ returns the result of $\text{PAEKS.Test}(\text{ct}_{\text{PAEKS}}, \text{td}'_{kw})$.

In the challenge phase, $\mathcal{A}$ declares (kw_0^*, kw_1^*) . $\mathcal{B}$ sends (kw_0^*, kw_1^*) to $\mathcal{C}$. $\mathcal{C}$ generates the challenge ciphertext $\text{ct}_{\text{PAEKS}}^* \leftarrow \text{PAEKS.Enc}(\text{pk}'_R, \text{pk}'_S, \text{sk}'_S, kw_\beta^*)$ and sends $\text{ct}_{\text{PAEKS}}^*$ to $\mathcal{B}$. $\mathcal{B}$ runs $\sigma \leftarrow \text{Sign}(\text{sigk}_S, \text{ct}_{\text{PAEKS}}^*)$ and $C \leftarrow \text{PKE.Enc}(\text{pk}_{\text{AS}}, \sigma || \text{ct}_{\text{PAEKS}}^*)$, and sends the challenge ciphertext $\text{ct}_{\text{DS-PAEKS}}^* = C$ to $\mathcal{A}$. Finally, $\mathcal{A}$ outputs β' . Then $\mathcal{B}$ outputs β' . If $\mathcal{A}$ breaks the IND-AS-CKA security, then $\mathcal{B}$ breaks the IND-CKA security with the same advantage. Thus, $|\Pr[E_3] - 1/2| \leq \text{Adv}_{\text{PAEKS}, \mathcal{B}}^{\text{IND-CKA}}(\lambda)$ holds. $\square$

Now, we have $|\Pr[E_0] - 1/2| \leq \text{Adv}_{\text{Sig}, \mathcal{B}}^{\text{sEUF-CMA}}(\lambda) + 2q_{\text{Trap}} \text{Adv}_{\text{PKE}, \mathcal{B}}^{\text{IND-CCA}}(\lambda) + \text{Adv}_{\text{PAEKS}, \mathcal{B}}^{\text{IND-CKA}}(\lambda)$. This concludes the proof of Theorem 1. $\square$

Theorem 2. *The proposed construction is IND-TS-CKA secure if PAEKS is IND-CKA secure.*

Proof Sketch. The adversary $\mathcal{A}$ is allowed to issue a transition query to $\mathcal{O}_{\text{Trans}}$ with no restriction. Thus, $\mathcal{A}$ can obtain $\text{int-ct}_{\text{DS-PAEKS}} = (\text{ct}_{\text{PAEKS}}, \sigma, \text{td}_{kw'})$ for any $\text{ct}_{\text{DS-PAEKS}} = C$. Moreover, $\mathcal{A}$ has the secret key of the test server. That is, $\mathcal{A}$ has $(\text{PK}', \text{DK}') \leftarrow \text{PKE.KeyGen}(1^\lambda)$ and can decrypt $\text{td}_{\text{DS-PAEKS}} = C'$ such that $\sigma' || \text{td}'_{kw'} \leftarrow \text{PKE.Dec}(\text{sk}_{\text{TS}}, \text{td}_{\text{DS-PAEKS}})$. Thus, $\mathcal{A}$ observes PAEKS ciphertexts and trapdoors directly. So, we directly reduce the IND-TS-CKA security to the IND-CKA security.

Theorem 3. *The proposed construction is IND-AS-IKGA secure if PKE is IND-CCA secure and Sig is sEUF-CMA secure.*

Proof Sketch. Basically, the IND-AS-IKGA security is reduced to the IND-CCA security of PKE because trapdoors are encrypted by the public key of the test server, and the adversary does not have the decryption key. To simulate the test oracle $\mathcal{O}_{\text{Test}}$, PKE is required to be IND-CCA secure because the decryption algorithm of PKE is internally run in the DS-PAEKS.Test algorithm. However, we must consider the following case: how to prevent any modification of DS-PAEKS ciphertexts of the challenge keyword because if an adversary issues $\text{int-ct}_{\text{DS-PAEKS}}$ to $\mathcal{O}_{\text{Test}}$, where $\text{int-ct}_{\text{DS-PAEKS}} \leftarrow \text{DS-PAEKS.Transition}(\text{pk}_R, \text{pk}_S, \text{pk}_{\text{AS}}, \text{sk}_{\text{AS}}, \text{ct}_{\text{DS-PAEKS}}, \text{td}_{\text{DS-PAEKS}}^*)$, $\text{ct}_{\text{DS-PAEKS}} \notin \text{CTSet}$, and $\text{ct}_{\text{DS-PAEKS}}$ is a DS-PAEKS ciphertext of the challenge keyword, then the adversary trivially wins. To handle the issue, we employ the sEUF-CMA security of the underlying signature scheme. That is, a PAEKS ciphertext is signed before encryption that prevents any modification of the DS-PAEKS ciphertext. Then, it is guaranteed that all DS-PAEKS ciphertexts $\mathcal{A}$ obtains are generated by the encryption oracle $\mathcal{O}_C$.

Theorem 4. *The proposed construction is IND-TS-IKGA secure if PAEKS is IND-IKGA secure.*

Proof Sketch. Since $\mathcal{A}$ has the secret key of the test server, $\mathcal{A}$ can decrypt $C' \leftarrow \text{PKE.Enc}(\text{pk}_{\text{TS}}, \sigma' || \text{td}'_{kw'})$ and can observe a PAEKS trapdoor $\text{td}'_{kw'}$ directly. Thus, as in IND-TS-CKA, we directly reduce the IND-TS-IKGA security to the IND-IKGA security.

7 Conclusion

In this paper, we propose a generic construction of DS-PAEKS derived from PAEKS, two PKE schemes, and two signature schemes. We also show that the DS-PAEKS scheme [4], the DS-PEKS scheme [5], and the DS-PEKS construction [18] are vulnerable.

Though our consistency definition follows that of PEKS defined by Abdalla et al. [1], there is a room for improving the security. For example, our consistency definition considers the case that a keyword for encryption and a keyword for trapdoor are different. However, a stronger definition has been considered in [10]. It considers a multi-sender setting, where a trapdoor associated with a sender does not work against ciphertexts generated by the secret key of another sender, even if the same keyword is associated. Moreover, an adversary is not allowed to issue any query. It seems natural to consider the case that an adversary declares keywords after observing some ciphertexts and trapdoors. Considering such stronger definitions in the DS-PAEKS context is left as a future work of this paper.

Acknowledgment: A part of study was done when the author was with the National Institute of Information and Communications Technology (NICT), Japan. This work was supported by JSPS KAKENHI Grant Number JP21K11897.

References

- [1] Michel Abdalla, Mihir Bellare, Dario Catalano, Eike Kiltz, Tadayoshi Kohno, Tanja Lange, John Malone-Lee, Gregory Neven, Pascal Paillier, and Haixia Shi. Searchable encryption revisited: Consistency properties, relation to anonymous IBE, and extensions. *Journal of Cryptology*, 21(3):350–391, 2008.
- [2] Dan Boneh, Giovanni Di Crescenzo, Rafail Ostrovsky, and Giuseppe Persiano. Public key encryption with keyword search. In *EUROCRYPT*, pages 506–522, 2004.
- [3] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT*, pages 514–532, 2001.
- [4] Biwen Chen, Libing Wu, Sherali Zeadally, and Debiao He. Dual-server public-key authenticated encryption with keyword search. *IEEE Transactions on Cloud Computing*, 10(1):322–333, 2022.
- [5] Rongmao Chen, Yi Mu, Guomin Yang, Fuchun Guo, and Xiaofen Wang. Dual-server public-key encryption with keyword search for secure cloud storage. *IEEE Transactions on Information Forensics and Security*, 11(4):789–798, 2016.
- [6] Yu-Chi Chen. SPEKS: secure server-designation public key encryption with keyword search against keyword guessing attacks. *The Computer Journal*, 58(4):922–933, 2015.
- [7] Leixiao Cheng and Fei Meng. Public key authenticated encryption with keyword search from LWE. In *ESORICS*, pages 303–324, 2022.

- [8] Leixiao Cheng and Fei Meng. Server-aided public key authenticated searchable encryption with constant ciphertext and constant trapdoor. *IEEE Transactions on Information Forensics and Security*, 19:1388–1400, 2024.
- [9] Tianyu Chi, Baodong Qin, and Dong Zheng. An efficient searchable public-key authenticated encryption for cloud-assisted medical internet of things. *Wireless Communications and Mobile Computing*, 2020:8816172:1–8816172:11, 2020.
- [10] Keita Emura. Generic construction of public-key authenticated encryption with keyword search revisited: Stronger security and efficient construction. In *ACM APKC*, pages 39–49, 2022.
- [11] Qiong Huang and Hongbo Li. An efficient public-key searchable encryption scheme secure against inside keyword guessing attacks. *Inf. Sci.*, 403:1–14, 2017.
- [12] Qinyi Li and Xavier Boyen. Public-key authenticated encryption with keyword search made easy. *IACR Commun. Cryptol.*, 1(2):16, 2024.
- [13] Long Meng, Liqun Chen, Yangguang Tian, Mark Manulis, and Suhui Liu. FEASE: fast and expressive asymmetric searchable encryption. In *USENIX Security*, 2024.
- [14] Mahnaz Noroozi and Ziba Eslami. Public key authenticated encryption with keyword search: revisited. *IET Information Security*, 13(4):336–342, 2019.
- [15] Baodong Qin, Yu Chen, Qiong Huang, Ximeng Liu, and Dong Zheng. Public-key authenticated encryption with keyword search revisited: Security model and constructions. *Inf. Sci.*, 516:515–528, 2020.
- [16] Baodong Qin, Hui Cui, Xiaokun Zheng, and Dong Zheng. Improved security model for public-key authenticated encryption with keyword search. In *ProvSec*, pages 19–38, 2021.
- [17] Hyun Sook Rhee, Jong Hwan Park, Willy Susilo, and Dong Hoon Lee. Improved searchable public key encryption with designated tester. In *ACM ASIACCS*, pages 376–379, 2009.
- [18] Raylin Tso, Kaibin Huang, Yu-Chi Chen, Sk. Md. Mizanur Rahman, and Tsu-Yang Wu. Generic construction of dual-server public key encryption with keyword search on cloud computing. *IEEE Access*, 8:152551–152564, 2020.

Appendix: Vulnerability of Previous DS-PEKS

Vulnerability of the Chen et al. DS-PEKS scheme. Our attack (given in Section 4) also works against the Chen et al. DS-PEKS scheme [5].³ The ciphertext form is $(g_1^{r_1}, g_2^{r_1}, h_1^{r_1} h_2^{r_1} H(kw))$ (now, no sender secret key is required for encryption). Here, the hash function H is defined as $H : \{0, 1\}^* \rightarrow \mathbb{G}$. In their security definition (SS-CKA: semantic-security against the chosen keyword attack, Fig. 1. in [5]), the oracle $\mathcal{O}_T$ is defined such that it takes a ciphertext and a keyword kw as input, and the oracle internally generates a trapdoor of kw and the intermediate ciphertext (internal testing state in [5]), and returns the result of the test algorithm. Here, $kw \notin$

³Here, we give an attack against the DDH-based construction given in [5]. However, our attack works against their generic construction from smooth projective hash functions.

$\{kw_0^*, kw_1^*\}$ is required.⁴ Thus, the same strategy above does not work. However, because of the malleability of the ciphertext, we can modify the challenge ciphertext as follows. Let $(C_1^*, C_2^*, C_3^*) = (g_1^{r_1^*}, g_2^{r_1^*}, h_1^{r_1^*} h_2^{r_1^*} H(kw_\beta^*))$ be the challenge ciphertext. The adversary computes $H(kw_0^*)$ and $H(kw)$ for arbitrary keyword $kw \notin \{kw_0^*, kw_1^*\}$. Then, the adversary chooses $r \xleftarrow{\$} \mathbb{Z}_p$ and computes $h_1^r h_2^r H(kw) C_3^* / H(kw_0^*) = H(kw) h_1^{r_1^*+r} h_2^{r_1^*+r} H(kw_\beta^*) / H(kw_0^*)$. If $\beta = 0$, then the ciphertext is an encryption of kw . If $\beta = 1$, then the ciphertext is an encryption of an unknown keyword (i.e., kw' where $H(kw') = H(kw)H(kw_1^*)/H(kw_0^*)$ holds). Here, kw is not equal to the unknown keyword because if the unknown keyword equals kw , then $H(kw)H(kw_1^*)/H(kw_0^*) = H(kw)$ holds and thus $H(kw_1^*) = H(kw_0^*)$. This contradicts the collision resistance of H because $kw_0^* \neq kw_1^*$. Thus, the adversary sends $(g_1^r C_1^*, g_2^r C_2^*, h_1^r h_2^r H(kw) C_3^* / H(kw_0^*))$ and kw to $\mathcal{O}_T$. If the oracle returns 1, then $\beta = 0$, and $\beta = 1$ otherwise. Thus, the adversary wins.

Vulnerability of the Tso et al. DS-PEKS scheme. Tso et al. [18] gave a semi-generic construction of DS-PEKS scheme from a PKE scheme. In their syntax, there are two servers, front server (assistant server in our notation) and back server (test server in our notation). Briefly, they employed a Pedersen commitment $g^r h^{kw}$ and encrypt X^r by using the underlying PKE scheme using the public key of the back server pk_{BS} , where $X = g^x$ is a public key of the front server. A ciphertext is described as $(g^r h^{kw}, \text{PKE.Enc}(\text{pk}_{BS}, X^r))$. A trapdoor has a similar form: $(g^{r'} h^{-kw'}, \text{PKE.Enc}(\text{pk}_{BS}, X^{r'}))$. The front server generates an intermediate ciphertext (they call it internal-testing-stage) using the secret key x such that $R((g^r h^{kw})(g^{r'} h^{-kw'}))^x = RX^{r+r'} h^{x(kw-kw')}$, where R is a random value. The intermediate ciphertext is described as $(\text{PKE.Enc}(\text{pk}_{BS}, X^r), \text{PKE.Enc}(\text{pk}_{BS}, X^{r'}), H(R), RX^{r+r'} h^{x(kw-kw')})$ where H is a hash function. If $kw = kw'$, then it is described as $(\text{PKE.Enc}(\text{pk}_{BS}, X^r), \text{PKE.Enc}(\text{pk}_{BS}, X^{r'}), H(R), RX^{r+r'})$. The back server decrypts $(\text{PKE.Enc}(\text{pk}_{BS}, X^r), \text{PKE.Enc}(\text{pk}_{BS}, X^{r'}))$, obtains $(X^r, X^{r'})$, and checks $H(RX^{r+r'} h^{kw-kw'} / X^r X^{r'}) = H(R)$ holds or not. If it holds, then output 1, and 0 otherwise. They claimed that information about keyword is perfectly hidden by g^r and $g^{r'}$.

The main problem here is that the PKE part is independent of the keyword to be searched and the CCA security of the PKE scheme is meaningless to hide information about keyword. Actually, due to the homomorphic property of the commitment part, an adversary $\mathcal{A}$ can know $\beta = 0$ or $\beta = 1$ as follows. Here, $\mathcal{A}$ is modeled as a malicious back server that has the secret key of the PKE scheme sk_{BS} and the public key of the front server X (but $\mathcal{A}$ does not know the secret key of the front server x) (See the definition of IND-CKA-BS in [18]). Let the challenge ciphertext and the challenge trapdoor be described as $c_\beta = (g^r h^{kw_\beta^*}, \text{PKE.Enc}(\text{pk}_{BS}, X^r))$ and $t_\beta = (g^{r'} h^{-kw_\beta^*}, \text{PKE.Enc}(\text{pk}_{BS}, X^{r'}))$. Note that, $\mathcal{A}$ declares the challenge keywords (kw_0^*, kw_1^*) , and the challenge ciphertext and the challenge trapdoor are given to the adversary simultaneously in their security model. $\mathcal{A}$ is allowed to access the front test oracle that takes a ciphertext $c \neq c_\beta$ and a trapdoor $t \neq t_\beta$, and returns the corresponding intermediate ciphertext. $\mathcal{A}$ prepares another ciphertext from c_β as follows. $\mathcal{A}$ decrypts $\text{PKE.Enc}(\text{pk}_{BS}, X^r)$ using sk_{BS} and obtains X^r . $\mathcal{A}$ randomly selects r'' and computes $g^{r''} g^r h^{kw_\beta^*} = g^{r''+r} h^{kw_\beta^*}$, $X^{r''} X^r = X^{r''+r}$, and $\text{PKE.Enc}(\text{pk}_{BS}, X^{r''+r})$. Now $c = (g^{r''+r} h^{kw_\beta^*}, \text{PKE.Enc}(\text{pk}_{BS}, X^{r''+r}))$ is a ciphertext of kw_β^* and $c \neq c_\beta$. Then, $\mathcal{A}$ generates a trapdoor t for kw_0^* and then $t \neq t_\beta$. Let r''' be used as the randomness. $\mathcal{A}$ sends (c, t) to the front test oracle, and obtains the corresponding intermediate ciphertext. The intermediate ciphertext is described as $(\text{PKE.Enc}(\text{pk}_{BS}, X^{r''+r}), \text{PKE.Enc}(\text{pk}_{BS}, X^{r'''}), H(R), RX^{r''+r+r'''} h^{x(kw_\beta^*-kw_0^*)})$.

⁴Tso et al. [18] have pointed out that the Chen et al. DS-PEKS scheme [5] is not as secure as they claimed. Basically, their attack is almost the same as ours, focusing on the linearity of smooth projective hash functions and using the test oracle. However, they generated another ciphertext of the challenge keyword from the challenge ciphertext, and sends the ciphertext and kw_1^* to the test oracle $\mathcal{O}_T$, that contradicts the restriction $kw \notin \{kw_0^*, kw_1^*\}$.

If $H(RX^{r''+r+r'''}h^{x(kw_\beta^*-kw_0^*)}/X^{r''+r}X^{r'''}) = H(R)$, then $\beta = 0$, and $\beta = 1$ otherwise. Thus, the adversary wins.